SESSION 10
BUILDING WEBSITES USING NODE.JS

Learning Objectives

In this session, students will learn to:

- Explain the process of creating a Web application using Node.js
- Describe the steps to connect the Web application to a MongoDB cloud database
- List the steps to upload the Web application to the GitHub repository
- Explain the procedure to deploy the Web application on Render from a GitHub repository

Node.js can be used to create a complete Website. The front-end Web application can be connected to a back-end database. GitHub and Render are crucial in the Web deployment process using Node.js.

This session begins with explaining the process of retrieving the connection string of a MongoDB cloud database. It then, progresses to describe the process of creating the Web application in Node.js. The session explains the procedure of uploading the Web application to the GitHub repository. It elaborates on the steps to deploy the application on Render from a GitHub repository. Finally, the session concludes by illustrating the functionality testing of the application in the Web browser.

10.1 Overview

Node.js can be used to create both front-end and back-end applications. In this session, a simple library application is created using Node.js and a

MongoDB cloud database called `Library`. Figure 10.1 shows the characteristics of the library application.

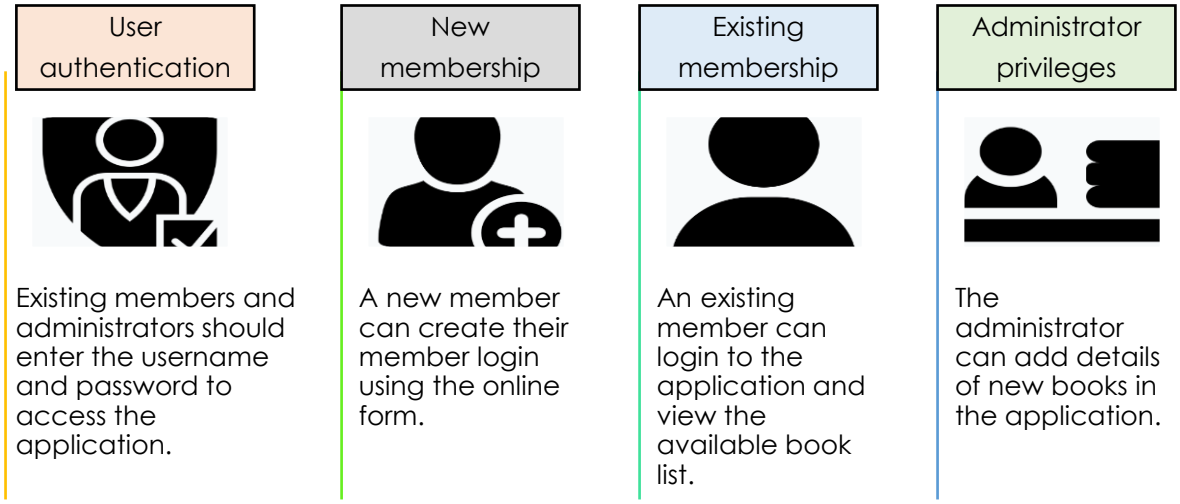
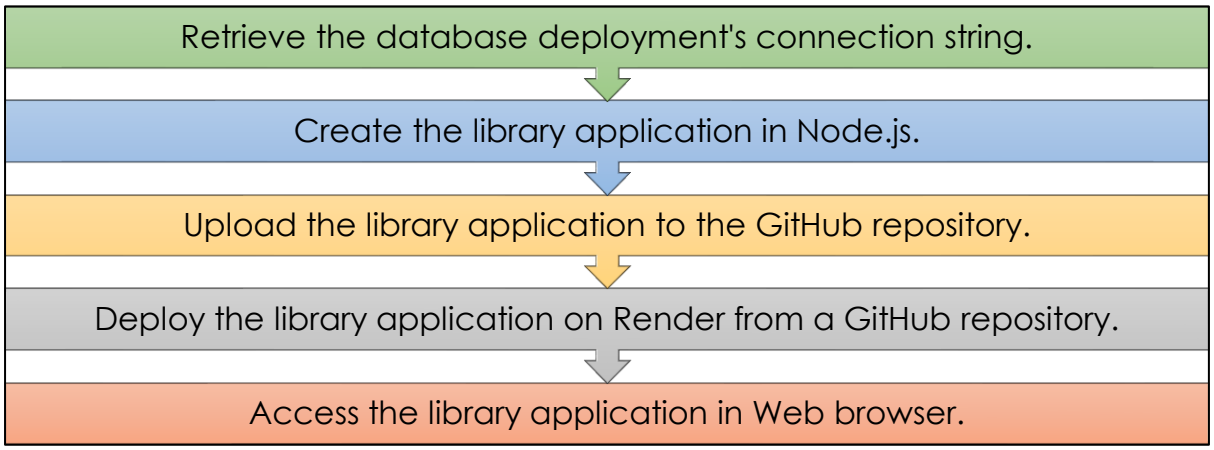


Figure 10.1: Library Application

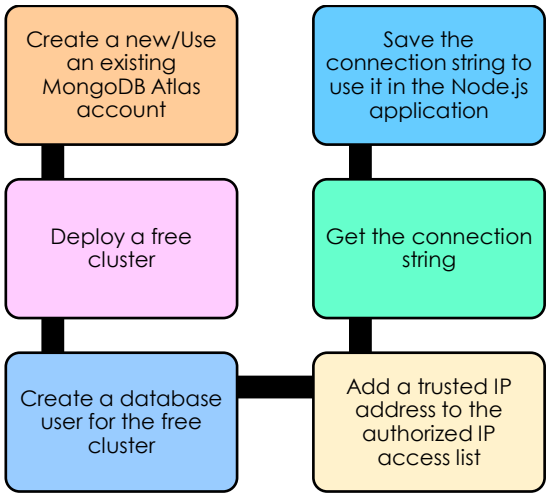
Steps in creating the library application are as follows:



10.2 Retrieving the Connection String from MongoDB Cloud

The prerequisite in creating the library application is to deploy a database in MongoDB cloud and retrieve the database deployment's connection string. The connection string is then used to connect the `Library` application with the cloud database for storing, retrieving, and processing data.

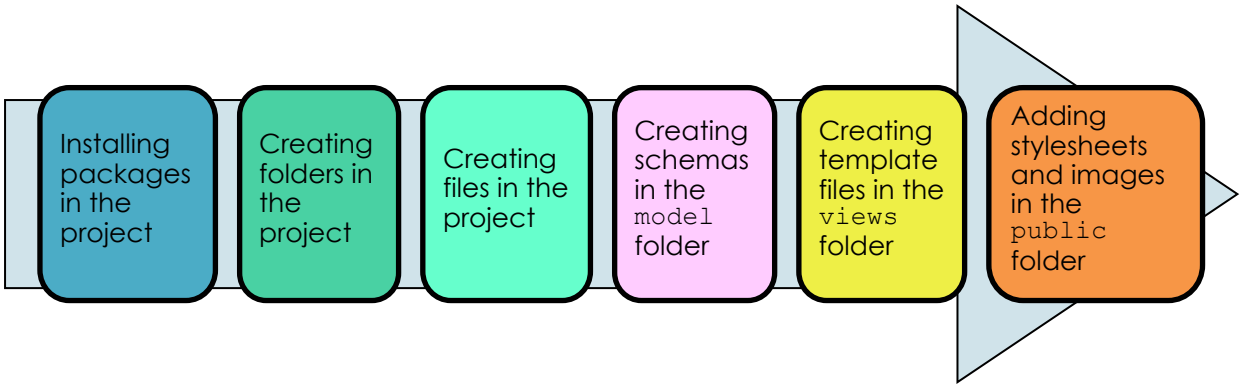
Here are the steps to be executed for achieving the task:



Refer to the Appendix A for a detailed description on how to obtain the connection string from the MongoDB cloud.

10.3 Creating the Library Application in Node.js

The next step is to create the library application in Node.js that involves following actions:



10.3.1 Installing Packages in the Project

To install the required Node.js packages, perform following tasks:

1. Create a directory for the application. To do so:
   a. Open Command Prompt.
   b. Navigate to the local drive where Node.js is installed.
   c. Navigate to the directory to create the application folder.
   d. Create a directory named `library_application` and change to that directory.

Figure 10.2 displays the directory path.

```
C:\Users\Linda\nodejs>cd library_application
```

Figure 10.2: Creation of a New Directory

2. Create the `package.json` file to initialize a new Node.js project for the application. Type the command at the prompt and press Enter.

```
npm init -y
```

The command creates the `package.json` file with the default values in the application's root directory, `library_application`. Figure 10.3 displays the output of the command.

```
C:\Users\Linda\nodejs\library_application>npm init -y
Wrote to C:\Users\Linda\nodejs\library_application\package.json:

{
  "name": "library_application",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

Figure 10.3: Creation of a New Node.js Project

The information in Figure 10.4 is stored in the `package.json` file as specified in the image.

3. Install the required packages in the application's directory.
Table 10.1 lists the purpose of the seven packages that must be installed in the application folder.

Package	Description
`passport`	Passport is a middleware in Node.js that offers authentication strategies using username-password, Twitter, Facebook, Instagram, and so on
`express`	A Web application framework that facilitates rapid application development
`ejs`	A JavaScript library for Embedded JavaScript (EJS) Templating used to generate Hypertext Markup Language (HTML) templates

Package	Description
mongoose	An Object Data Modeling (ODM) library for MongoDB used to facilitate communication with the MongoDB database and perform database operations
body-parser	A Node.js middleware used to parse the body of HTTP POST requests
express-session	A session middleware to manage the session data
passport-local	A package used to implement user authentication process
passport-local-mongoose	A mongoose plug-in that helps in user authentication and session management

Table 10.1: Packages to be Added to the Project

All these packages can be installed using a single command. To install the packages, type the command at the prompt and press Enter:

```
npm install passport express ejs mongoose body-parser
express-session passport-local passport-local-mongoose
```

Figure 10.4 displays the output of the command.

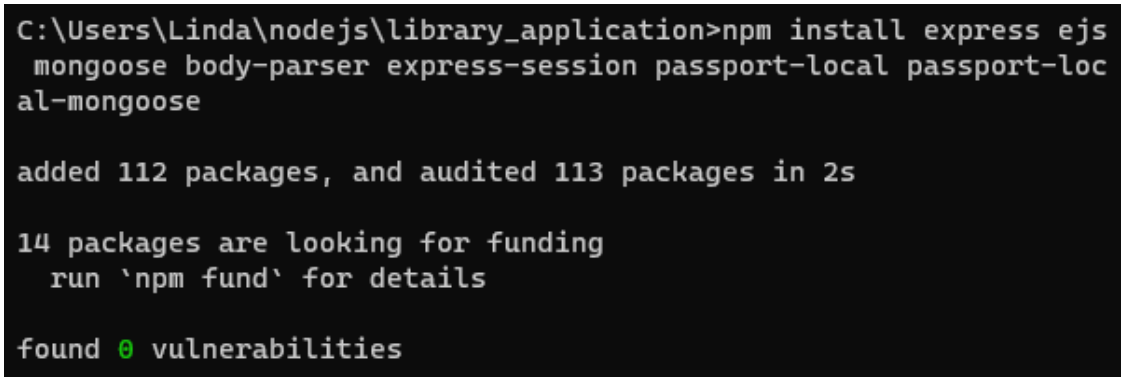


Figure 10.4: Installation of Packages

After installing the packages, observe the structure of the `library_application` folder as shown in Figure 10.5.

```
C:\Users\Linda\nodejs\library_application>dir
 Volume in drive C is Windows
 Volume Serial Number is 66C5-B39B

 Directory of C:\Users\Linda\nodejs\library_application

11/30/2023  06:56 PM    <DIR>          .
11/30/2023  06:36 PM    <DIR>          ..
11/30/2023  06:58 PM    <DIR>          node_modules
11/30/2023  06:58 PM            43,889 package-lock.json
11/30/2023  06:58 PM               467 package.json
               2 File(s)         44,356 bytes
               3 Dir(s)  275,915,255,808 bytes free
```

Figure 10.5: Listing of the `library_application` Folder

The `node_modules` folder and `package-lock.json` file are automatically created when `npm` command is executed to install various packages in the project. The `package-lock.json` file includes the complete details of all the packages installed in the project. It should not be manually edited. Also, the `package.json` file is automatically updated with the version numbers of the installed packages as shown in Figure 10.6.

Figure 10.6: `package.json` File With Version Numbers

4. Open the `package.json` file in VS Code. Edit the file to update the value of the `main` field, and to include the `engines` and `node` fields as shown in Code Snippet 1.

Code Snippet 1:

```
{
 "name": "library_app",
 "version": "1.0.0",
```

```
 "description": "",
 "main": "app.js",
 "scripts": {
 "test": "echo \"Error: no test specified\" && exit 1"
 },
 "keywords": [],
 "author": "",
 "license": "ISC",
 "dependencies": {
 "body-parser": "^1.20.2",
 "ejs": "^3.1.9",
 "express": "^4.18.2",
 "express-session": "^1.17.3",
 "mongoose": "^8.0.0",
 "passport": "^0.6.0",
 "passport-local": "^1.0.0",
 "passport-local-mongoose": "^8.0.0"
 },
 "engines": {
 "node": "20.7.0"
 }
}
```

Save and close the updated `package.json` file.

10.3.2 Creating Folders in the Project

To organize the project structure, create the required folders and subfolders as shown in Figure 10.7.

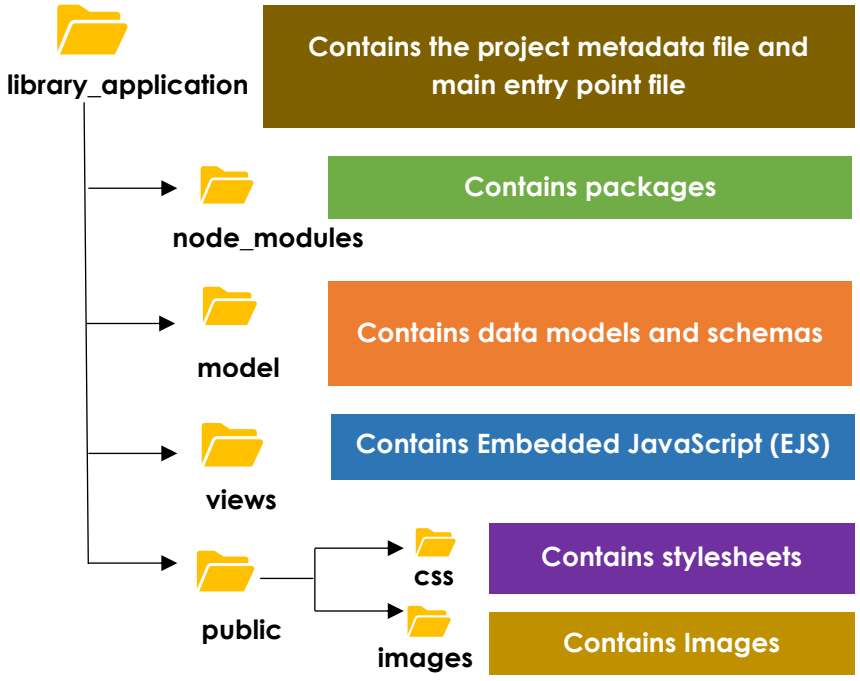


Figure 10.7: Project Folder Structure

Figure 10.8 shows the listing of the `library_application` folder.

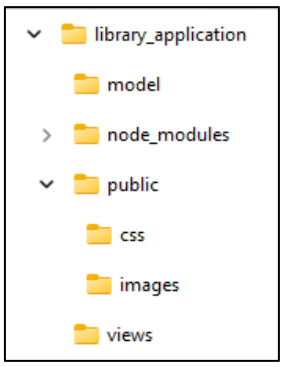


Figure 10.8: Listing of the `library_application` Folder

10.3.3 Creating Files in the Project

To define the user interface design, logic, and functionality of the project, create the required source code files under the specified project folders.

- **Creating Schemas in the `model` Folder**

To define the schemas for the `Library` database collections, create two models, `User.js` and `Book.js`, in the `model` folder. The schemas represent the structure of the `books` and `users` collections that are to be stored in the `Library` database. Models enable the developers to create, update, and retrieve data in the application.

1. **Create `User.js`**

   Code Snippet 2 shows the code for `User.js`. Save the code with the file name `User.js` in the `model` folder.

   Code Snippet 2:

```
const mongoose = require('mongoose')
const Schema = mongoose.Schema
const passportLocalMongoose = require('passport-local-
mongoose');
var User = new Schema({
 username: {
 type: String
 },
 password: {
 type: String
 }
})
User.plugin(passportLocalMongoose);
module.exports = mongoose.model('User', User)
```

Operations demonstrated in the `User.js` file are as follows:

- The `mongoose` package is imported and used for interacting with the MongoDB database.
- The statement `const Schema = mongoose.Schema` imports the `Schema` class from the `mongoose` package. The `Schema` class is used to define the structure of the collections that will be stored in the database.
- The statement `var User = new Schema({ })` defines the schema for the `User` model. The schema specifies two fields, `username` and `password`, and their data types.
- The plugin `passportLocalMongoose` is added to the `User` schema for authentication purposes.
- `mongoose.model('User', User)` creates a `model` object called `User` based on the `User` schema. Mongoose creates a collection based on the `User` model. It automatically assigns the name of the collection as `users`, which is a plural, lowercased version of the model's name. The `module.exports` statement exports the `User` model as a module. Exporting the `model` object enables the developers to access the `User` model in any other file or project.

2. Create Book.js

Code Snippet 3 creates a `Book` model for storing book details in the `Library` database. The code creates a `Schema` object called `bookSchema`. It then creates a `model` called `Book` based on the `bookSchema`. As models represent collections, Mongoose creates a collection based on the `Book` model in the `Library` database and automatically assigns the name of the collection as `books`. The code exports the `Book` model to make it accessible in another file or project.

Save the code with the file name `Book.js` in the `model` folder.

Code Snippet 3:

```
const mongoose = require("mongoose");
const bookSchema = new mongoose.Schema({
 Book_id: Number,
 Book_name: String,
 Author_name: String,
 Price: String, // Assuming price is a number
 Age_group: String,
 Book_type: String,
});
const Book = mongoose.model("Book", bookSchema);
module.exports = Book;
```

After creating the schema files, the structure of the `model` folder appears as shown in Figure 10.9.

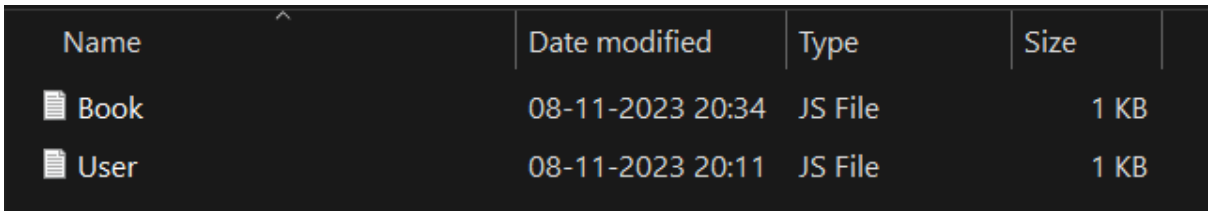


Figure 10.9: Listing of the `model` Folder

- **Creating Template Files in the `views` Folder**

To render HTML pages for the application, create the template files or `.ejs` files in the `views` folder. The `.ejs` files are used to provide the HTML markups by embedding the required JavaScript in the main Node.js application file, `app.js`.

1. **Create `register.ejs`**

   Code Snippet 4 demonstrates the HTML code for the registration form for new library members. The completed form, when submitted, will be delivered to the `/register` endpoint for further processing. Save the code with the file name `register.ejs` in the `views` folder.

   Code Snippet 4:

```
<!DOCTYPE html>
<html>
<head>
 <link rel="stylesheet" type="text/css"
href="/css/register.css">
 <title>Sign up for Library Member</title>
</head>
<body>
 <div class="container">
     <img src="/images/book1.jpeg" alt="Image 1">
 <h1>Sign up form for Library Member</h1>
 <form action="/register" method="POST" class="form-
table">
 <table>
 <tr>
 <td><label for="username">Username</label></td>
 <td><input type="text" name="username" id="username"
placeholder="Username" autocomplete="off"></td>
 </tr>
 <tr>
 <td><label for="password">Password</label></td>
```

```
 <td><input type="password" name="password"
id="password" placeholder="Password"></td>
 </tr>
 <tr>
 <td></td>
 <td><button>Sign-UP</button></td>
 </tr>
 </table>
 </form>
     <p><a href="/logout">Home Page</a></p>
 </div>
</body>
</html>
```

2. **Create `login.ejs`**

Code Snippet 5 demonstrates the HTML code for the application's login form. The completed form, when submitted, will be delivered to the `/login` endpoint for further processing. Save the code with the file name `login.ejs` in the `views` folder.

Code Snippet 5:

```
<!DOCTYPE html>
<html>
<head>
 <link rel="stylesheet" type="text/css"
href="/css/register.css">
 <title>Member Login</title>
</head>
<body>
 <div class="container">
     <img src="/images/book1.jpeg" alt="Image 1">
 <h1>Member login</h1>
 <form action="/login" method="POST">
 <table>
 <tr>
 <td><label for="username">Username</label></td>
 <td><input type="text" name="username" id="username"
placeholder="Username" autocomplete="off"></td>
 </tr>
 <tr>
 <td><label for="password">Password</label></td>
 <td><input type="password" name="password"
id="password" placeholder="Password"></td>
 </tr>
 <tr>
 <td></td>
 <td><button>Login</button></td>
 </tr>
```

```
 </table>
 </form>
     <p><a href="/logout">Home Page</a></p>
 </div>
</body>
</html>
```

3. Create `home.ejs`

Code Snippet 6 demonstrates the HTML code for the application's home page. On the home page of the application, the user can:

- Sign up as a new member
- Login as an existing member
- Login as an administrator

Save the code with the file name `home.ejs` in the `views` folder.

Code Snippet 6:

```
<!DOCTYPE html>
<html>
<head>
 <link rel="stylesheet" type="text/css"
href="/css/home.css">
 <title>Member Login</title>
</head>
<body>
<div class="container">
<div class="right">
<h1>Library Home page</h1>
<table align="center" style="width:50%">
<tr>
 <td align="center"><a href="/register">Sign_up for
Member User</a></td>
 </tr>
<tr>
 <td align="center"><a href="/login">Member
Login</a></td>
 </tr>
<tr>
 <td align="center"><a href="/admin">Administrator
Login</a></td>
 </tr>
</table>
</div>
<div class="left">
 <img class="center-image" src="/images/book1.jpeg"
alt="Image 1">
 </div>
```

```
</div>
</body>
</html>
```

4. **Create `admin-login.ejs`**

Code Snippet 7 demonstrates the HTML code for the administrator's login page. Administrators can enter their username and password in the login form. The completed form, when submitted, will be sent to the `/admin-login` endpoint for further processing.

Save the code with the file name `admin-login.ejs` in the `views` folder.

Code Snippet 7:

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" type="text/css"
href="/css/register.css">
 <title>Admin Login</title>
</head>
<body>
 <div class="container">
     <img src="/images/book1.jpeg" alt="Image 1">
 <h1>Admin Login</h1>
 <form action="/admin-login" method="post">
 <table>
 <tr>
 <td><label for="username">Username</label></td>
 <td><input type="text" name="username" id="username"
placeholder="Username" autocomplete="off"></td>
 </tr>
 <tr>
 <td><label for="password">Password</label></td>
 <td><input type="password" name="password"
id="password" placeholder="Password"></td>
 </tr>
 <tr>
 <td></td>
 <td><button>Login</button></td>
 </tr>
 </table>
 </form>
<p><a href="/logout">Home Page</a></p>
 </div>
</body>
</html>
```

5. **Create `admin-dashboard.ejs`**

Code Snippet 8 demonstrates the HTML code for the admin dashboard page. On this page, the administrators can add new book details to the library's `books` collection. On clicking the **Add Book** button, the completed form will be sent to the `/admin-dashboard/add-book` endpoint for further processing.

Save the code with the file name `admin-dashboard.ejs` in the `views` folder.

Code Snippet 8:

```
<!DOCTYPE html>
<html>
<head>
 <link rel="stylesheet" type="text/css"
href="/css/admindash.css">
 <title>Admin Dashboard</title>
</head>
<body>
 <div class="container">
 <form action="/admin-dashboard/add-book"
method="post">
 <table>  <tr>
 <td><img src="/images/book1.jpeg" alt="Image 1"
class="left-image"></td>
 <td><h1>Create Book Detail</h1></td>
 </tr>
 <tr>
 <td><label for="Book_id">BookID:</label></td>
 <td><input type="number" name="Book_id" required
autocomplete="off"></td>
 </tr>
 <tr>
 <td><label for="Book_name">Book Name:</label></td>
 <td><input type="text" name="Book_name" required
autocomplete="off"></td>
 </tr>
 <tr>
 <td><label for="Author_name">Author
Name:</label></td>
 <td><input type="text" name="Author_name" required
autocomplete="off"></td>
 </tr>
 <tr>
 <td><label for="Price">Price:</label></td>
 <td><input type="text" name="Price" required
autocomplete="off"></td>
 </tr>
```

```
 <tr>
 <td><label for="Age_group">Age Group:</label></td>
 <td><input type="text" name="Age_group" required
autocomplete="off"></td>
 </tr>
 <tr>
 <td><label for="Book_type">Book Type:</label></td>
 <td><input type="text" name="Book_type" required
autocomplete="off"></td>
 </tr>
 </table>
 <div class="center-button-container">
 <button class="center-button" type="submit">Add
Book</button>
 </div>
 </form>
 </div>
 <p><a href="/logout">Logout</a></p>
</body>
</html>
```

6. **Create `booklist.ejs`**

Code Snippet 9 demonstrates the HTML code of a view that displays the list of books retrieved from the `books` collection.

The use of embedded JavaScript (`<% %>`) shows that:

- Data is retrieved and displayed dynamically by a server-side application.
- Book data is injected into the HTML template at runtime.

Save the code with the file name `booklist.ejs` in the `views` folder.

Code Snippet 9:

```
<!DOCTYPE html>
<html>
<head>
     <link rel="stylesheet" type="text/css"
href="/css/bookstyle.css">
 <title>List of Books</title>
</head>
<body>
<div class="content">
<div class="container">
 <div class="column1">
 <h1>List of Books</h1>
 <table class="center">
<tr>
```

```
 <th>Book ID</th>
 <th>Book </th>
 <th>Author </th>
 <th>Price</th>
 <th>Age Group</th>
 <th>Book Type</th>
 </tr>
 <% books.forEach(function(book) { %>
          <tr>
          <td><%= book.Book_id %></td>
          <td><%= book.Book_name %></td>
          <td><%= book.Author_name %></td>
          <td><%= book.Price %></td>
          <td> <%= book.Age_group %></td>
          <td> <%= book.Book_type %></td>
     </tr>
 <% }); %>
 </table>
 </div>
</div>
</div>
<p><a href="/logout">Logout</a></p>
</body>
</html>
```

7. Create `error.ejs`

Code Snippet 10 demonstrates the HTML code for the error page of member logins.
The error page:

- Displays error messages to members.
- Provides an option to return to the login page.

Save the code with the file name `error.ejs` in the `views` folder.

Code Snippet 10:

```
<!-- error.ejs -->
<!DOCTYPE html>
<html>
<head>
     <link rel="stylesheet" type="text/css"
href="/css/error.css">
 <title>Error</title>
</head>
<body>
<div class="container">
 <h1>Error</h1>
 <p><%= errorMessage %></p>
```

```
 <p><a href="/login">Go back to login</a></p>

</div>
</body>
</html>
```

8. Create admin-`error.ejs`

Code Snippet 11 demonstrates the HTML code for the error page of administrator login. The error page:

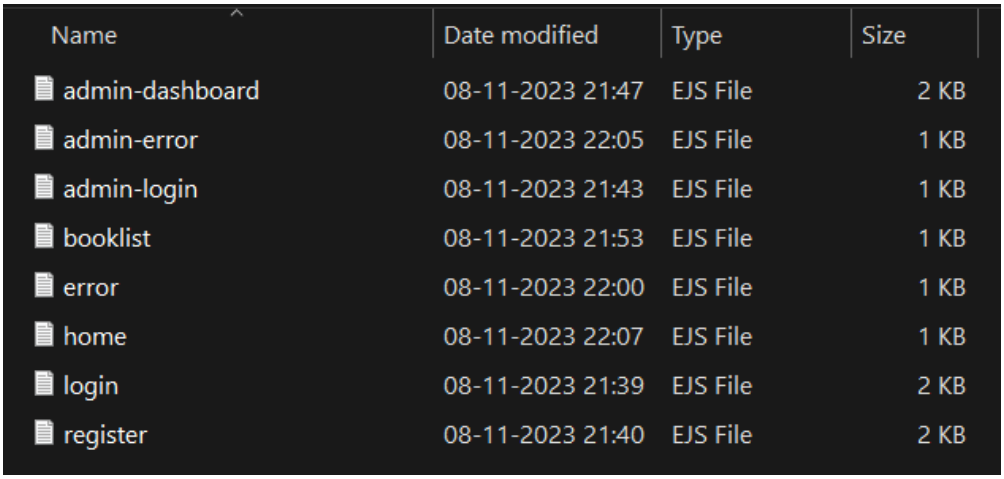
- Displays error messages to administrators
- Provides an option to return to the admin login page

Save the code with the file name `admin-error.ejs` in the `views` folder.

Code Snippet 11:

```
<!-- error.ejs -->
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" type="text/css"
href="/css/error.css">
 <title>Error</title>
</head>
<body>
<div class="container">

 <h1>Error</h1>
 <p><%= errorMessage %></p>
 <p><a href="/admin">Go back to admin login</a></p>
</div>
</body>
</html>
```

After the creation of `.ejs` files, find the structure of the `views` folder as shown in Figure 10.10.



Name	Date modified	Type	Size
admin-dashboard	08-11-2023 21:47	EJS File	2 KB
admin-error	08-11-2023 22:05	EJS File	1 KB
admin-login	08-11-2023 21:43	EJS File	1 KB
booklist	08-11-2023 21:53	EJS File	1 KB
error	08-11-2023 22:00	EJS File	1 KB
home	08-11-2023 22:07	EJS File	1 KB
login	08-11-2023 21:39	EJS File	2 KB
register	08-11-2023 21:40	EJS File	2 KB

Figure 10.10: Listing of the `views` Folder

- **Adding Stylesheets and Images in the `public` Folder**

Next, include the required `.css` and image files in the `public\css` and `public\images` folders, respectively.

1. **Create `register.css`**

   Code Snippet 12 shows the stylesheet for the new member registration form. Save the code with the file name `register.css` in the `public\css` folder.

   Code Snippet 12:

```
/* register.css */
body {
 margin: 0;
 padding: 0;
 font-family: Arial, sans-serif;
 background-color: lightblue; /* Background color for
the entire page */
}
h1 {
 border: 2px #eee solid;
 color: brown;
 text-align: center;
 padding: 10px;
 margin: 0; /* Add this line to remove any default
margin */
}
.container {
 display: flex;
 flex-direction: column; /* Change to a column layout
*/
 justify-content: center;
```

```
 align-items: center;
 height: 100vh;
 text-align: center; /* Center-align the content */
}
.form-table {
 background-color: #fff; /* Background color for the
form container */
 padding: 20px;
 border-radius: 8px;
 box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}
table {
 border-collapse: collapse;
 width: 100%;
}

table td {
 padding: 10px;
}

input {
 width: 100%;
 padding: 10px;
 margin: 5px 0;
 border: 1px solid #ccc;
 border-radius: 4px;
}

button {
 width: 100%;
 padding: 10px;
 background-color: #007BFF; /* Button background color
*/
 color: #fff; /* Button text color */
 border: none;
 border-radius: 4px;
 cursor: pointer;
}

button:hover {
 background-color: #0056b3; /* Button background color
on hover */
}
```

2. Create `home.css`

Code Snippet 13 shows the stylesheet for the application's home page. Save the code with the file name `home.css` in the `public\css` folder.

Code Snippet 13:

```
table, th, td {
 border: 1px solid;
 font-size: 30px;
}

body {
 background-color: lightblue;
}
h1 {
 border: 2px #eee solid;
 color: brown;
 text-align: center;
 padding: 10px;
}
.center-image {
 display: block;
 margin: 0 auto;
 max-width: 100%;
 height: auto;
}

.container {
 height: 75%;
 overflow: hidden;
 }
.right {
 width: 800px;
 float: right;
 margin-top: 100px;
 }
 .left {
 margin-top: 125px;
 width: auto;
 overflow: hidden;
 }
```

3. Create `error.css`

Code Snippet 14 shows the stylesheet for the application's error pages. Save the code with the file name `error.css` in the `public\css` folder.

Code Snippet 14:

```
.container {
 display: flex;
 flex-direction: column;
 align-items: center;
 width: 100%; /* Set the container width to 100% */
```

```
 max-width: 1000px; /* Optionally, set a maximum width
for the container */
 margin: 0 auto; /* Center the container horizontally
*/
}
body {
 margin: 0;
 padding: 0;
 font-family: Arial, sans-serif;
 background-color: lightblue;
 display: flex;
 flex-direction: column;
 align-items: center;
 justify-content: flex-start;
 min-height: 100vh;
}
```

4. Create `bookstyle.css`

Code Snippet 15 shows the stylesheet for displaying the book details. Save the code with the file name `bookstyle.css` in the `public\css` folder.

Code Snippet 15:

```
container {
      display: grid;
      grid-template-columns: 100px 150px 200px;
      text-align: center;
     }
.column1 {
 float: left;
 width: 90%;
 padding: 10px;
 height: auto;
 background-color: lightblue;
}
body {
 background-color: grey;
}
.content {
 width: 100%;
 position: absolute;
 top: 10%;
 }
.center {
 display: table;
 height: 100%;
 width: 100%;
}
h2 {
```

```
 border: 2px #eee solid;
 color: Red;
 text-align: center;
 padding: 10px;
}
```

5. Create `admindash.css`

Code Snippet 16 shows the stylesheet for administrator dashboard. Save the code with the file name `admindash.css` in the `public\css` folder.

Code Snippet 16:

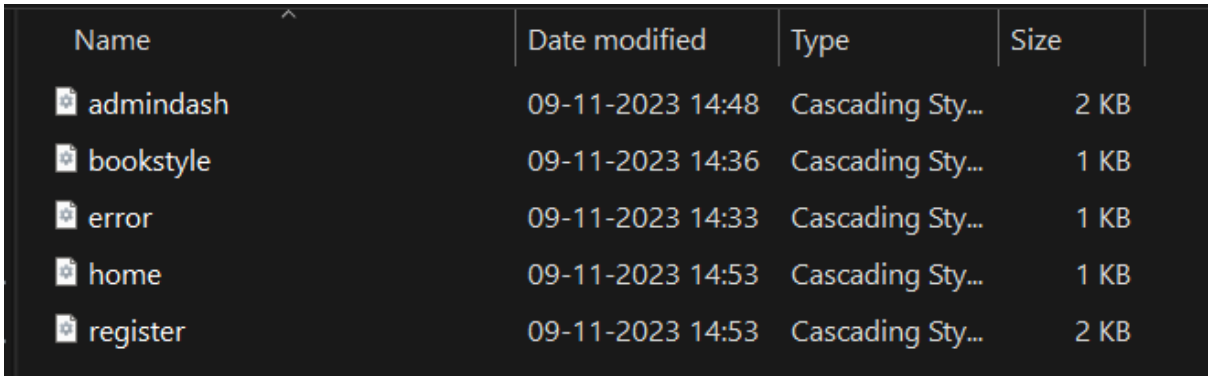
```
/* admin-dashboard.css */
body {
 margin: 0;
 padding: 0;
 font-family: Arial, sans-serif;
 background-color: lightblue;
 display: flex;
 flex-direction: column;
 align-items: center;
 justify-content: center; /* Center both horizontally
and vertically */
 min-height: 100vh;
}

.container {
 display: flex;
 flex-direction: column;
 align-items: center;
 width: 100%;
 max-width: 1000px;
 margin: 0 auto;
}

h1 {
 border: 2px #eee solid;
 color: brown;
 text-align: center;
 padding: 10px;
 margin: 0;
}

form {
 width: 60%;
}

table {
 border-collapse: collapse;
```

```
 width: 100%;
 margin: 20px auto;
}

table td {
 padding: 10px;
}

label {
 font-weight: bold;
 display: block;
 text-align: left;
}

input {
 width: 100%;
 padding: 10px;
 margin: 5px 0;
 border: 1px solid #ccc;
 border-radius: 4px;
}



.center-button-container {
 text-align: center; /* Center horizontally */
 margin-top: 20px; /* Adjust the top margin as
required */

}

.center-button {
 width: 150px; /* Adjust the width as required */
 height: 50px; /* Adjust the height as required */
 background-color: #007BFF; /* Blue background color
*/
 color: #fff;
 border: none;
 cursor: pointer;
}
```

6. Add Images

Download the image file, `book1.jpeg`, from https://commons.wikimedia.org/wiki/File:Books_HD_%288314929977%29.jpg and upload it to the `public\images` folder.

After the creation of `.css` files, find the structure of the `public\css` folder as shown in Figure 10.11.



Name	Date modified	Type	Size
admindash	09-11-2023 14:48	Cascading Sty...	2 KB
bookstyle	09-11-2023 14:36	Cascading Sty...	1 KB
error	09-11-2023 14:33	Cascading Sty...	1 KB
home	09-11-2023 14:53	Cascading Sty...	1 KB
register	09-11-2023 14:53	Cascading Sty...	2 KB

Figure 10.11: Listing of the `public\css` Folder

- **Creating the `app.js` File**

Finally, create the main Node.js application file, `app.js`. This file acts as the main entry point in the library application. In this file, the core logic of the application is handled. Routes and middleware are also defined.

Consider the code in Code Snippet 17 for `app.js` file.

Code Snippet 17:

```
const express = require("express");
const mongoose = require("mongoose");
const passport = require("passport");
const bodyParser = require("body-parser");
const LocalStrategy = require("passport-local");
const passportLocalMongoose = require("passport-local-
mongoose");
const User = require("./model/User");
const Book = require("./model/Book");
const app = express();
mongoose.connect(
`mongodb+srv://lindalarrissa91:linda91@cluster0.gktucwf.mongo
db.net/Library?retryWrites=true&w=majority`
);
app.set("view engine", "ejs");
app.use(bodyParser.urlencoded({ extended: true }));
app.use(
 require("express-session")({
 secret: "Rio is a dog",
 resave: false,
 saveUninitialized: false,
 })
);
app.use(passport.initialize());
app.use(passport.session());
app.use(express.static(__dirname + "/public"));
```

```
// Add the admin local strategy
passport.use(
 "admin-local",
 new LocalStrategy(function (username, password, done) {
 if (username === "Admin" && password === "12345") {
 return done(null, { username: "Aptech" });
 }
 return done(null, false, {message: "Incorrect admin username
or password" });
 })
);
passport.serializeUser(function (user, done) {
 // Here, you might want to serialize user data if required
(e.g., user.id)
 done(null, user);
});

passport.deserializeUser(function (user, done) {
 // Implement deserialization logic here if required
 done(null, user);
});
// Showing home page
app.get("/", function (req, res) {
 res.render("home");
});
// Showing register form
app.get("/register", function (req, res) {
 res.render("register");
});

// Handling user signup
app.post("/register", async (req, res) => {
 const user = await User.create({
 username: req.body.username,
 password: req.body.password,
 });
 res.redirect("/");
});

// Showing login form
app.get("/login", function (req, res) {
 res.render("login");
});

// Handling user login
app.post("/login", async function (req, res) {
 try {
 const user = await User.findOne({ username:
req.body.username });
 if (user) {
 const result = req.body.password === user.password;
```

```
 if (result) {
 const books = await Book.find({});
 res.render("booklist", { books: books });
 } else {
 res.render("error", { errorMessage: "Password doesn't match"
});
 }
 } else {
 res.render("error", { errorMessage: "User doesn't exist" });
 }
 } catch (error) {
 res.render("error", { errorMessage: "An error occurred" });
 }
});

// Admin login route
app.get("/admin", function (req, res) {
 res.render("admin-login");
});
// Admin login form
app.post(
 "/admin-login",
 passport.authenticate("admin-local", {
 successRedirect: "/admin-dashboard",
 failureRedirect: "/admin-error",
 })
);
// Admin error route
app.get("/admin-error", function (req, res) {
 res.render("admin-error", { errorMessage: "Incorrect admin
username or password" });
});
// Admin dashboard route
app.get("/admin-dashboard", function (req, res) {
 if (req.isAuthenticated()) {
 res.render("admin-dashboard");
 } else {
 res.redirect("/admin");
 }
});
// Get book details from the form
app.post("/admin-dashboard/add-book", function (req, res) {
 if (req.isAuthenticated()) {
 const bookDetails = {
 Book_id: req.body.Book_id,
 Book_name: req.body.Book_name,
 Author_name: req.body.Author_name,
 Price: req.body.Price,
 Age_group: req.body.Age_group,
 Book_type: req.body.Book_type,
 };
```

```
 // Create a new book in the "books" collection
 Book.create(bookDetails)
 .then((newBook) => {
 console.log("Book added successfully:", newBook);
 res.redirect("/admin-dashboard");
 })
 .catch((err) => {
 console.error("Failed to add the book:", err);
 res.status(500).json({ error: "Failed to add the book" });
 });
 } else {
 res.redirect("/admin");
 }
});
// Handling user logout
app.get("/logout", function (req, res) {
 req.logout(function (err) {
 if (err) {
 return next(err);
 }
 res.redirect("/");
 });
});
function isLoggedIn(req, res, next) {
 if (req.isAuthenticated()) return next();
 res.redirect("/login");
}
const port = process.env.PORT || 3000;
app.listen(port, function () {
 console.log("Server Has Started!");
});
```

Save the code with the file name `app.js` in the `library_application` folder. The operations demonstrated in the `app.js` file are as follows:

- **Importing packages**: Various Node.js packages such as `express`, `mongoose`, `passport`, `body-parser`, `passport-local`, and `passport-local-mongoose` are imported.
- **Importing schemas**: Two data models namely, `User` and `Book` from the local `model` directory are imported.
- **Creating `express` application**: The statement `const app = express()` creates the `express` application.
- **Connecting the database**: The statement `mongoose.connect(` `` `mongodb+srv://lindalarrissa91:linda91@cluster0.gktucwf.mongodb.net/Library?retryWrites=true&w=majority` `` `)` connects the application to the MongoDB database, `Library`, using the connection string.
- **Setting the view engine**: The statement `app.set("view engine", "ejs")` is used to set the view engine to EJS.

- **Setting up the middleware**: Packages such as `body-parser`, `express-session`, and `passport-authenticate` are used to set up the middleware.
- **Defining the local strategy**: A local strategy is defined for authenticating the administrators. If the username `Admin` and password `12345` match, the function returns an administrator object. Else, it returns an error message.
- **Serializing and deserializing user functions**: Functions are defined for serializing and deserializing user objects. These functions are used to manage user sessions.
- **Defining routes**: The application defines several routes for handling HTTP requests. The routes are:
    - `/`: This is the home page route.
    - `/register`: This is for registering a new member.
    - `/login`: This is for authenticating an existing member.
    - `/admin`: This is the administrator login route.
    - `/admin-login`: This is for handling administrator username and password.
    - `/admin-error`: This is for displaying administrator login errors.
    - `/admin-dashboard`: This is the administrator dashboard route.
    - `/admin-dashboard/add-book`: This is for handling the addition of a new book by the administrator.
    - `/logout`: This is the member/administrator logout route.
- **Handling new member registration**: On the `/register` route, new member registration is handled with the provided username and password. The username and password are added to the `users` collection in the `Library` database.
- **Handling existing member login**: On the `/login` route, the existing member login is handled. It checks if the member exists and whether the password matches. If successful, it retrieves a list of books from the database and displays the `booklist` from the `views` folder. Else, it displays an error message.
- **Handling administrator login and dashboard**: The administrator login is handled with a separate strategy using the administrator username and password. If the authentication is successful, the administrator is redirected to the `admin dashboard`. If not, an error message is displayed.
- **Managing `admin dashboard` and adding books**: The `admin dashboard` route is protected by authentication. Administrators can add new books by submitting a form. These books are added to the `books` collection in the `Library` database and the administrator is redirected to the dashboard.
- **Managing member/administrator logout process**: The member can log out by accessing the `/logout` route, which clears the session and redirects them to the home page.

- **Listening for requests**: The application listens on a specified port, either from the environment variable or port 3000. A message is logged when the server starts.

This completes the coding for the application.

10.4 Uploading the Library Application to the GitHub Repository

The developed application must be loaded to the Internet for it to be accessible globally. For this purpose, the GitHub repository is used. It can store codes, files, version histories, and so on, of applications. To upload the library application to the GitHub repository, create an exclusive repository space in GitHub.

Refer to Appendix B for a detailed description on how to:
- Create a repository in GitHub account
- Upload the application resources to the repository

10.5 Deploying the Library Application on Render from a GitHub Repository

Render is a cloud computing platform that helps developers to deploy their applications on cloud. The library application is uploaded to the GitHub repository. To deploy the application on Render from the GitHub repository, perform the given steps:

1. To access the **Render** platform, in the browser, navigate to the URL https://render.com/, and click **GET STARTED FOR FREE** as shown in Figure 10.12.

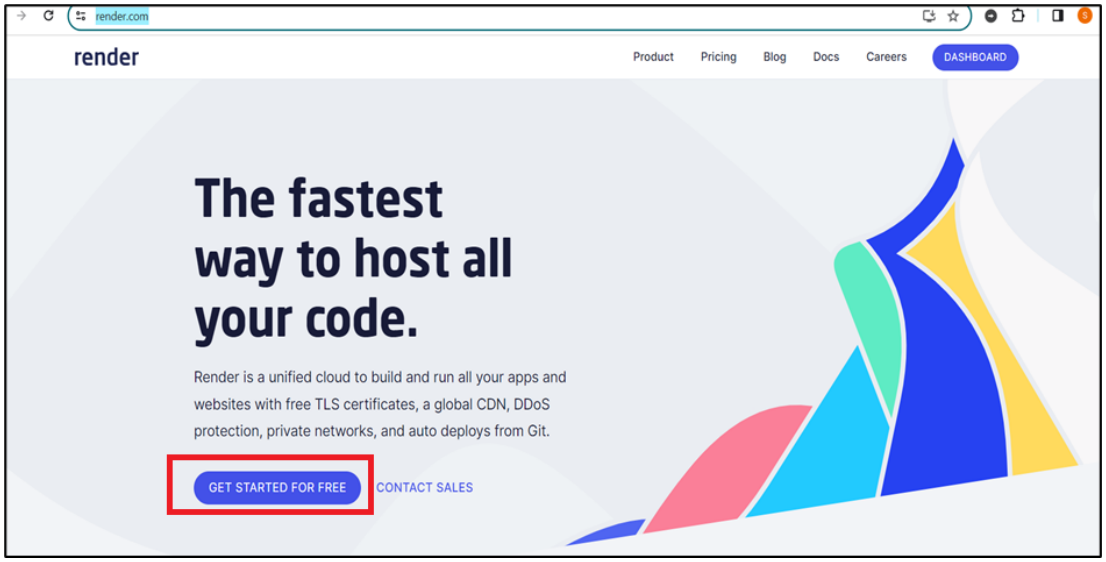


Figure 10.12: Home Page of Render

2. To sign in with the GitHub account, on the **Sign up for Render** page, type the given details, and click **Sign in** as shown in Figure 10.13.

   Here, for example, a GitHub account is used to sign into Render with the email as Linda.larrissa91@gmail.com and password as **linda91@**.

   - In the **Email** box, type the Gitbub account id.
   - In the **Password** box, type the password.

   If an existing account in used to sign in to Render, then provide the email id, password and click **Sign in**.

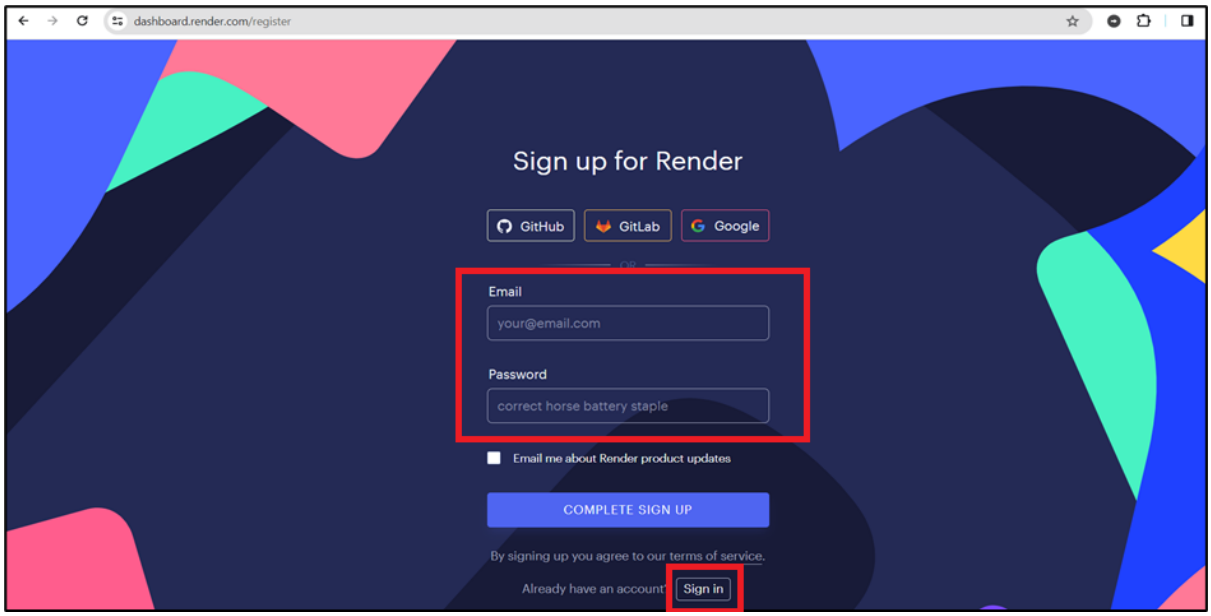


Figure 10.13: Sign up for Render Page

3. To complete the sign up process, on the **Sign up for Render** page, click **COMPLETE SIGN UP** as shown in Figure 10.14.

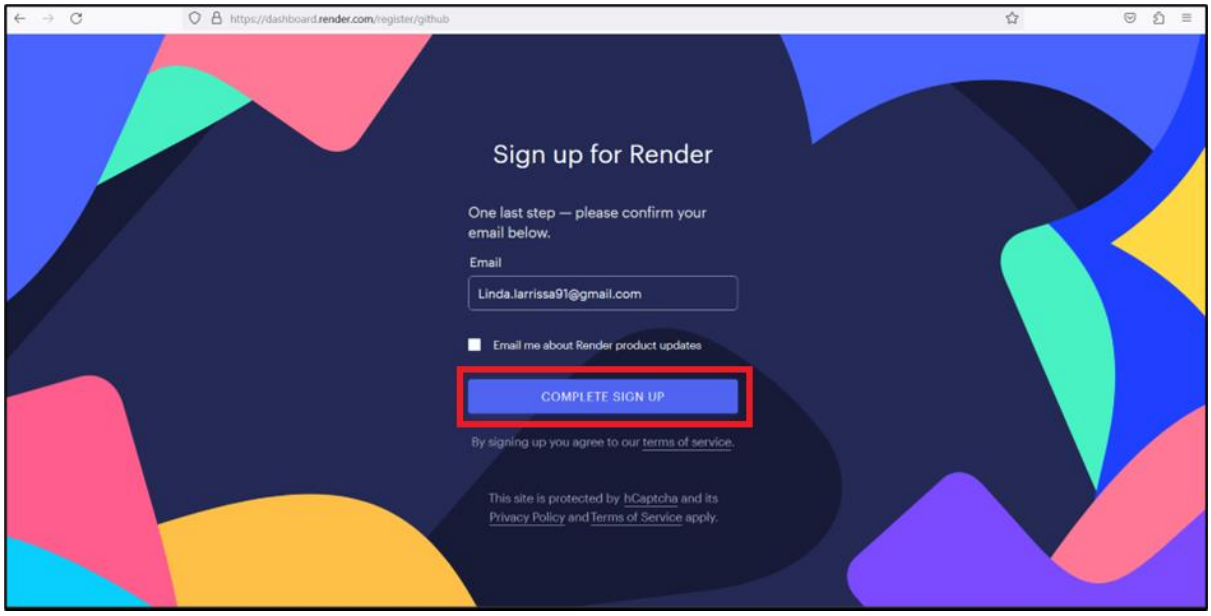


Figure 10.14: Sign up for Render Page – Email Confirmation

4. To activate the **Render** account, click the link in the email as instructed on the **Almost there** page as shown in Figure 10.15.

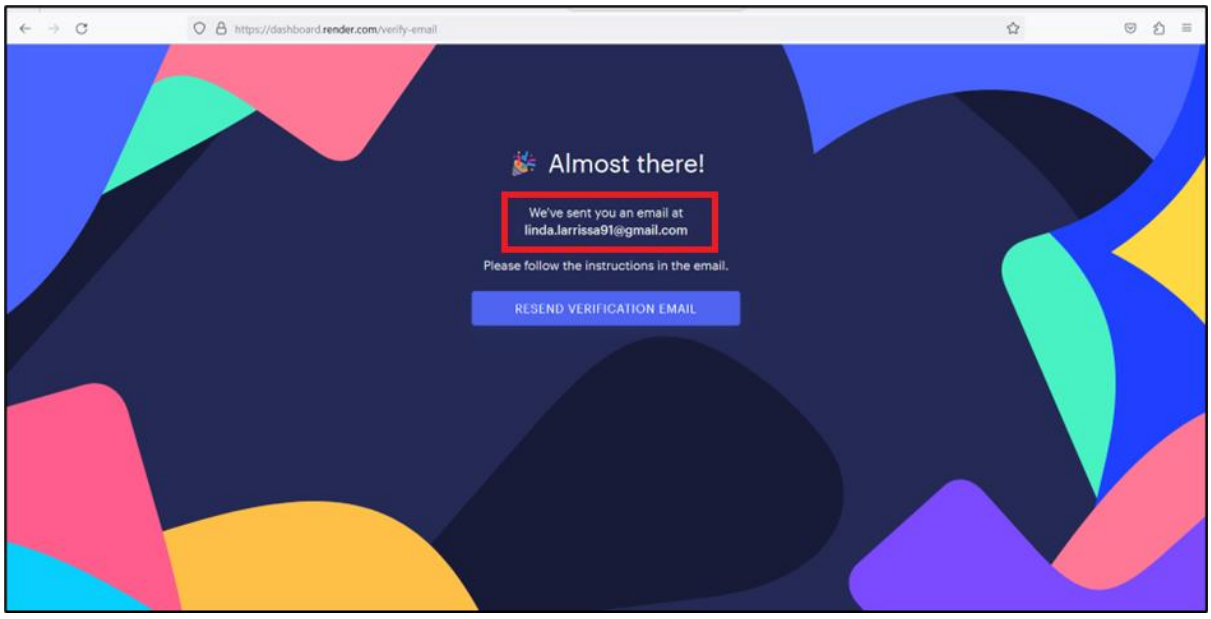


Figure 10.15: Almost there Page

5. To continue on **Render**, on the **Tell us about yourself** page, fill in the necessary details, and click **CONTINUE TO RENDER** as shown in Figure 10.16.

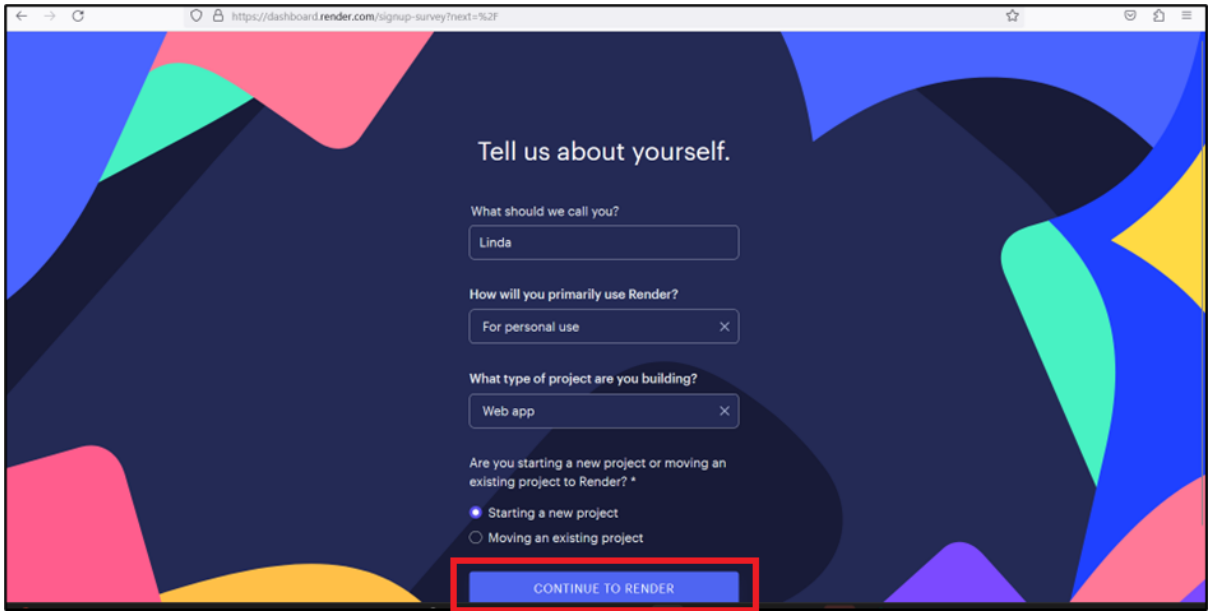


Figure 10.16: Tell us about yourself Page

6. To create a new Web service, on the **Render Dashboard** page, under **Web Services**, click **New Web Service** as shown in Figure 10.17.

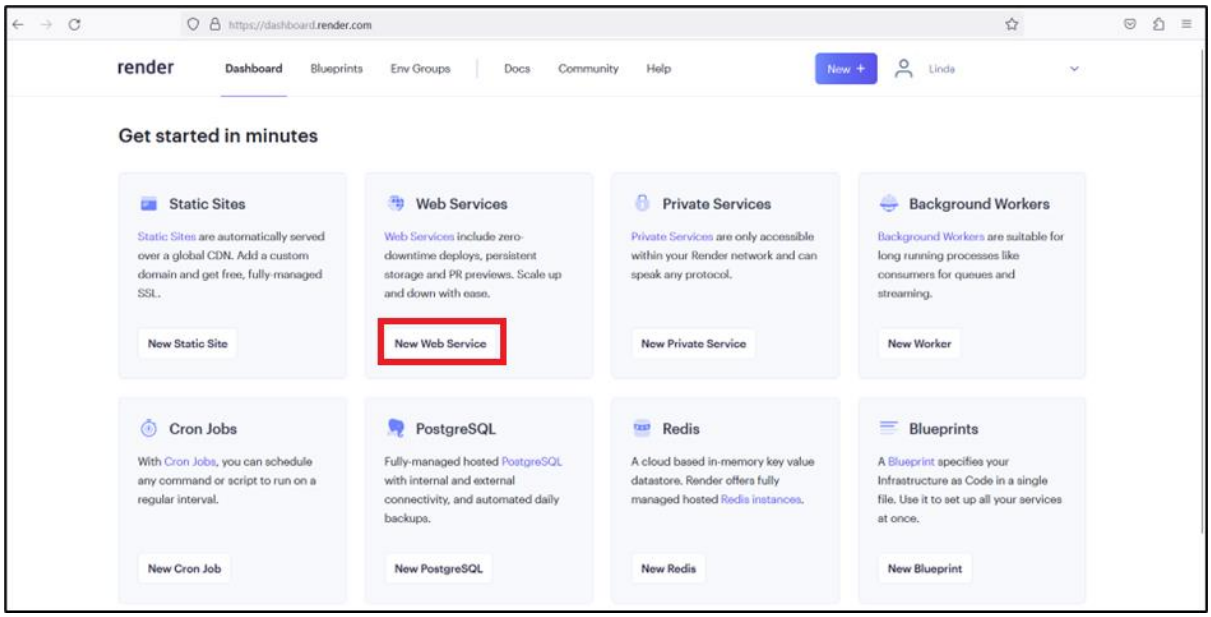


Figure 10.17: Render Dashboard Page

7. To deploy the Web service, on the **Create a new Web Service** page, click **Build and deploy from a Git repository**, and click **Next** as shown in Figure 10.18.

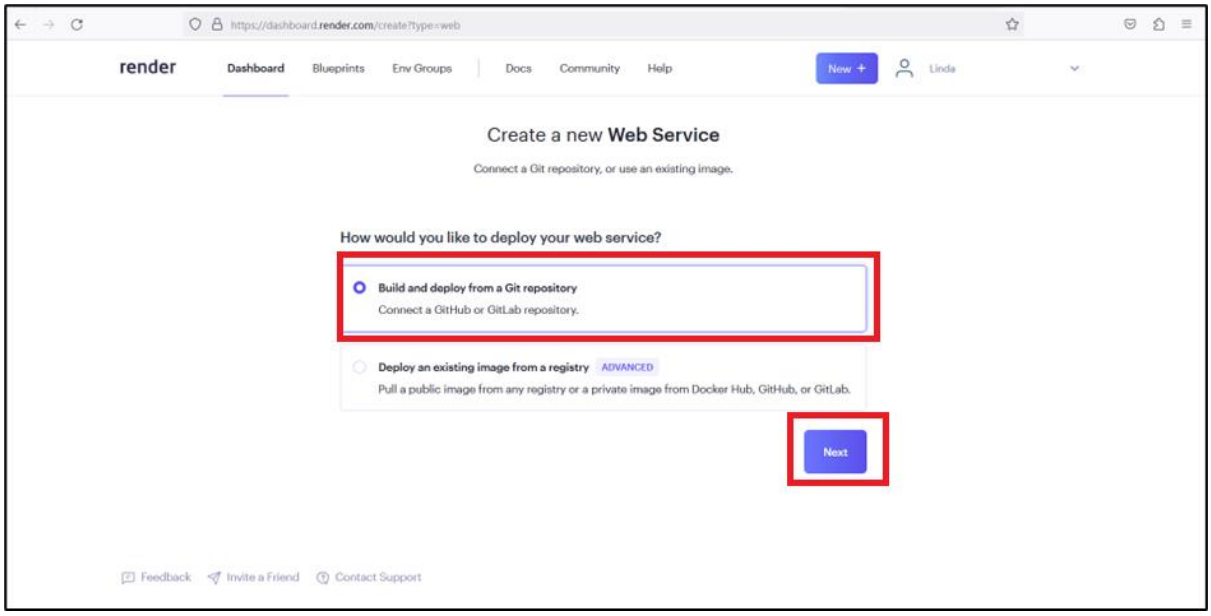


Figure 10.18: Create a new Web Service Page

8. To connect a repository, on the **Connect a repository** page, click **Connect GitHub** as shown in Figure 10.19.

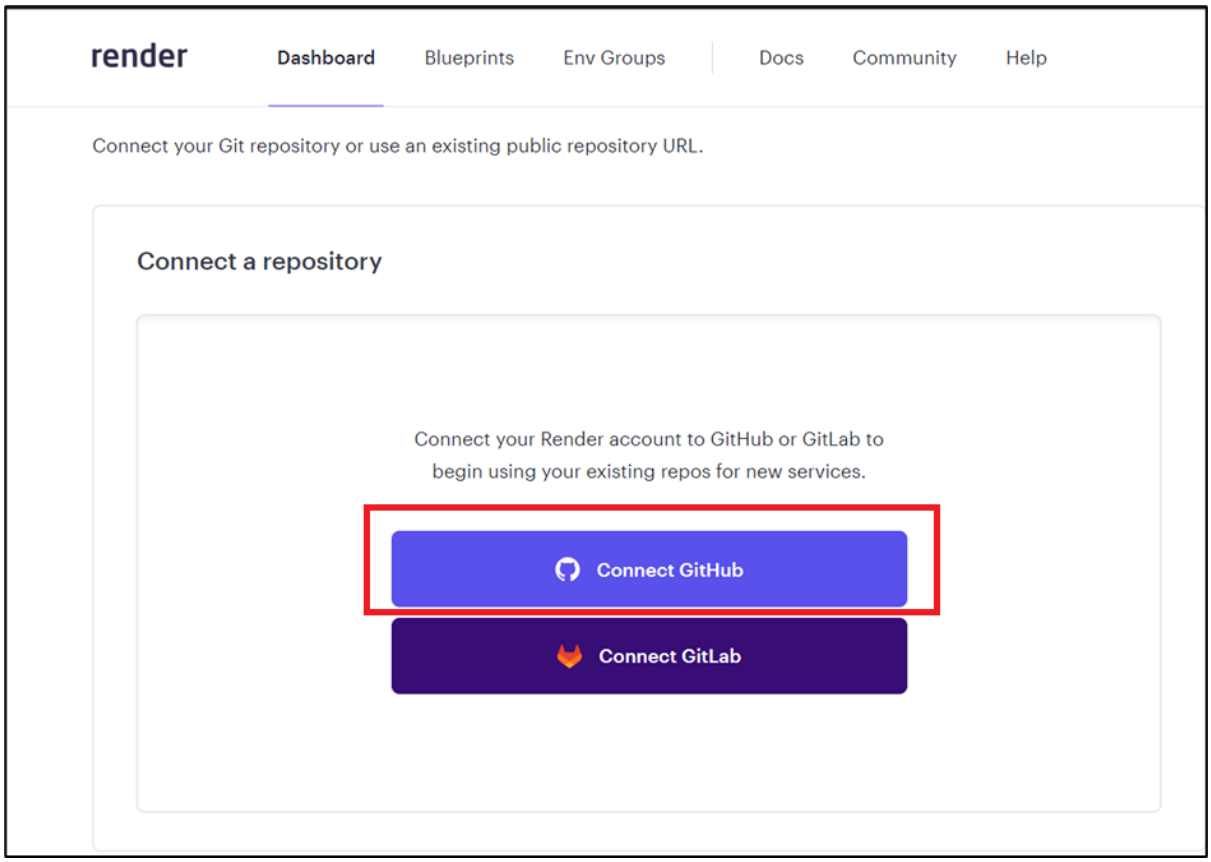


Figure 10.19: Connect a Repository Page

9. To authorize Render, on the **Render by Render would like permission to** page, click **Authorize Render** as shown in Figure 10.20.

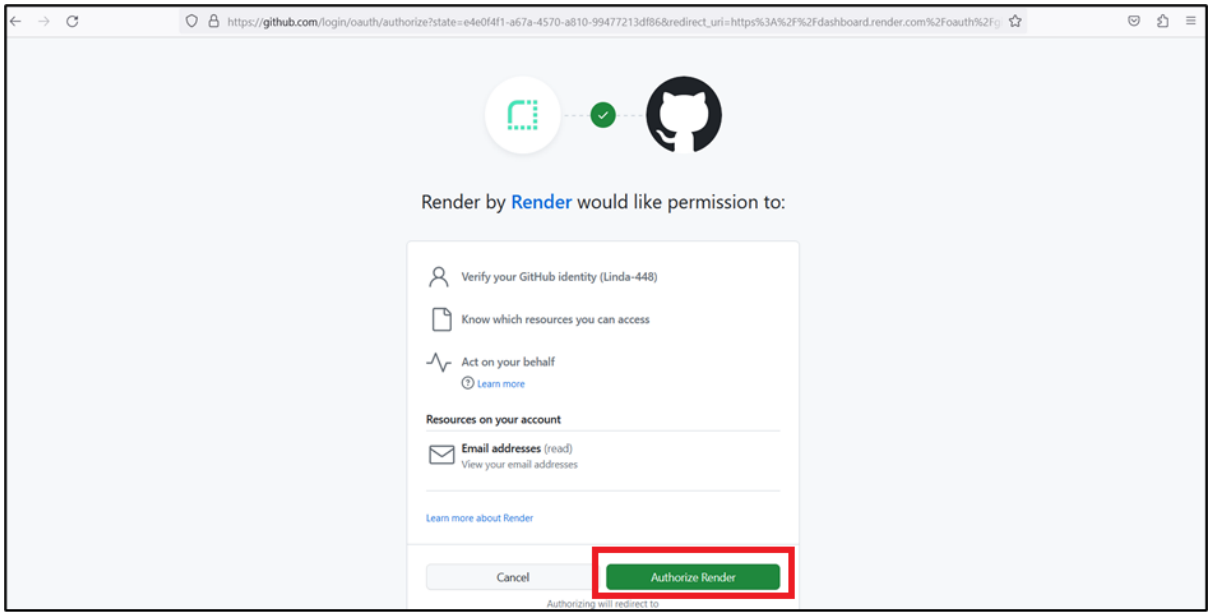


Figure 10.20: Render by Render would like permission to Page

10. To install **Render** on the GitHub account, on the **Install Render** page, click **Install** as shown in Figure 10.21.

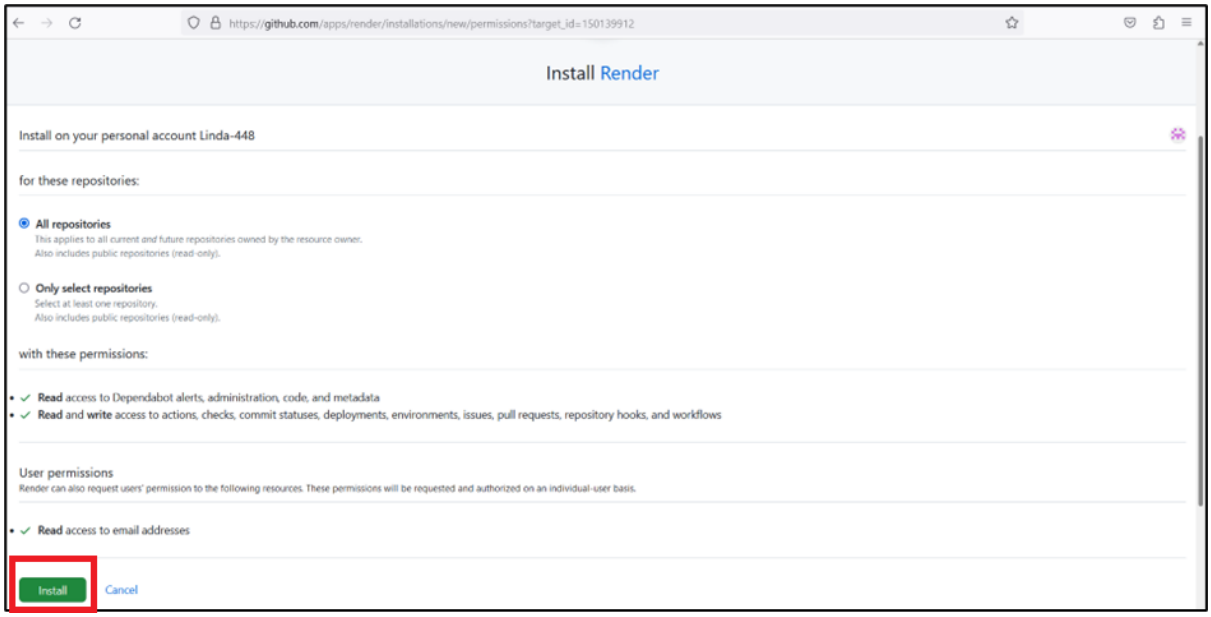


Figure 10.21: Install Render Page

11. To connect the repository, on the **Create a new Web Service** page, click **Connect** as shown in Figure 10.22.

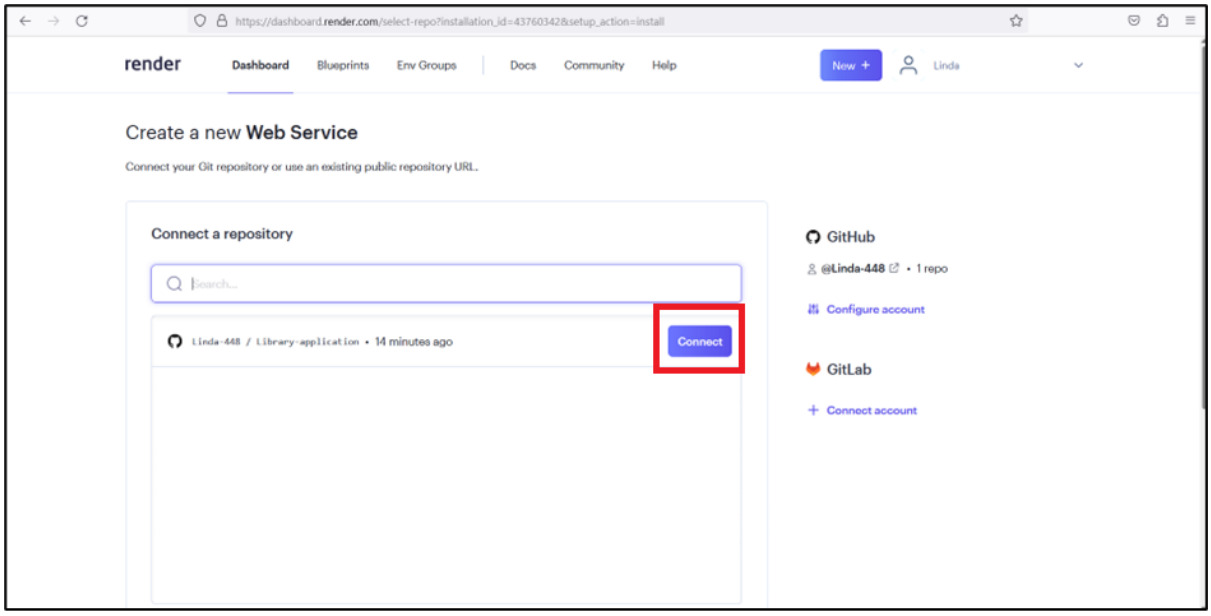


Figure 10.22: Connect the Repository

12. To provide a unique name for the Web service, on the **Render Dashboard** page, in the **Name** box, type **Library-app** as shown in Figure 10.23.

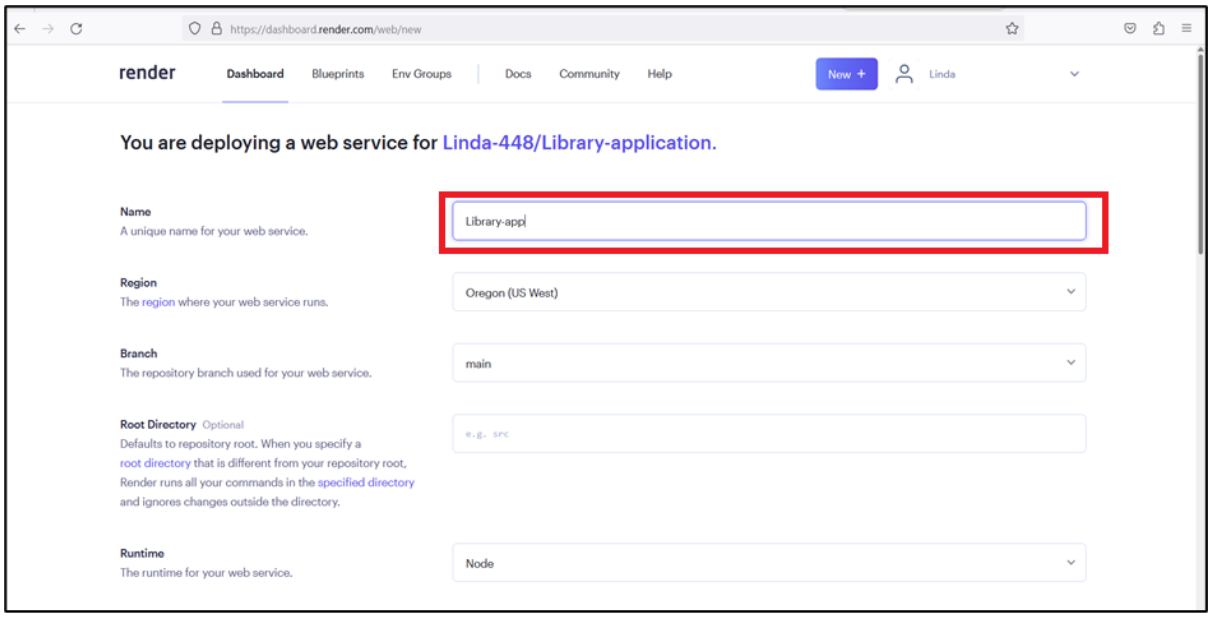


Figure 10.23: Web Service Name

13. To provide the start command, on the **Render Dashboard** page, scroll down and in the **Start Command** box, type **node app.js** as shown in Figure 10.24.

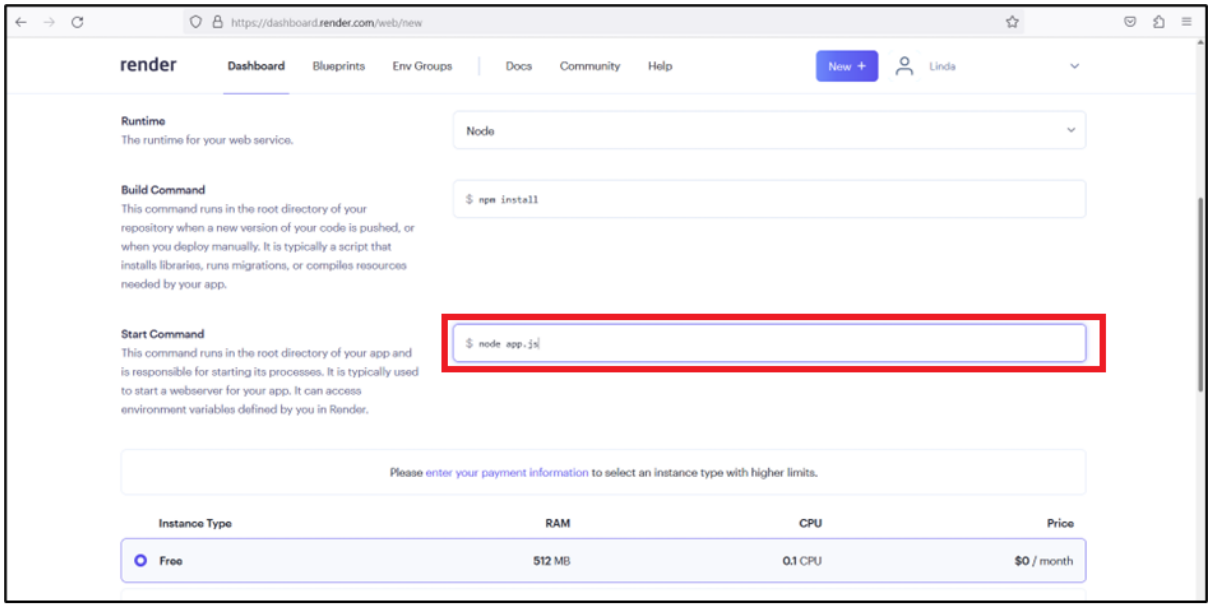


Figure 10.24: Start Command for Web Service

14. To create the Web service, on the **Render Dashboard** page, scroll down, and click **Create Web Service** as shown in Figure 10.25.

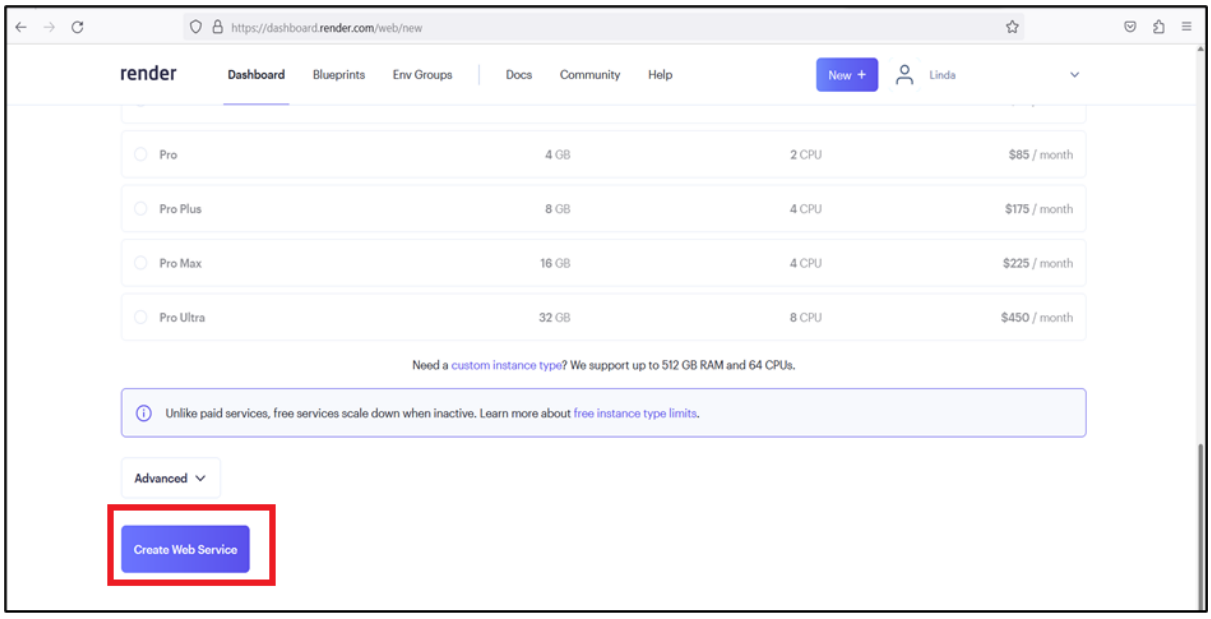


Figure 10.25: Create Web Service on Render Dashboard Page

15. Notice that the **Library-app** is deployed and the server has started as shown in Figure 10.26.

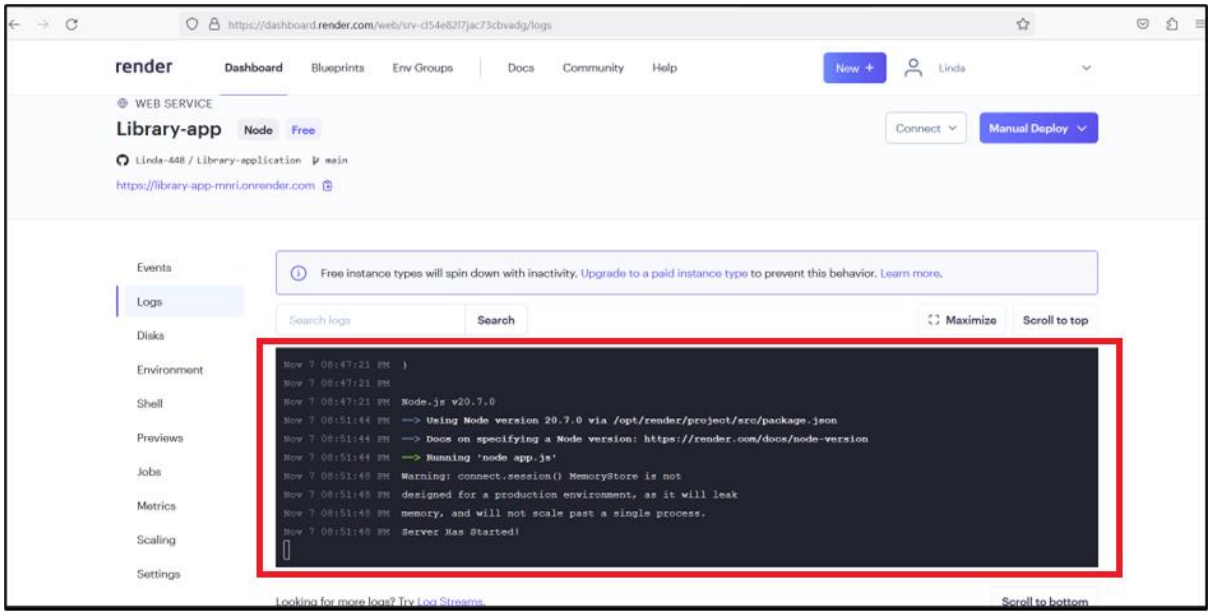


Figure 10.26: Deployment of Application

16. To identify the IP address of Render for the Library application, on the **Render Dashboard** page, click the **Connect** drop-down list as shown in Figure 10.27.

    The static outbound IP addresses are displayed. It is not safe to use the IP address 0.0.0.0/0, which was specified while connecting the application to MongoDB database. Therefore, use any one IP address from the list.

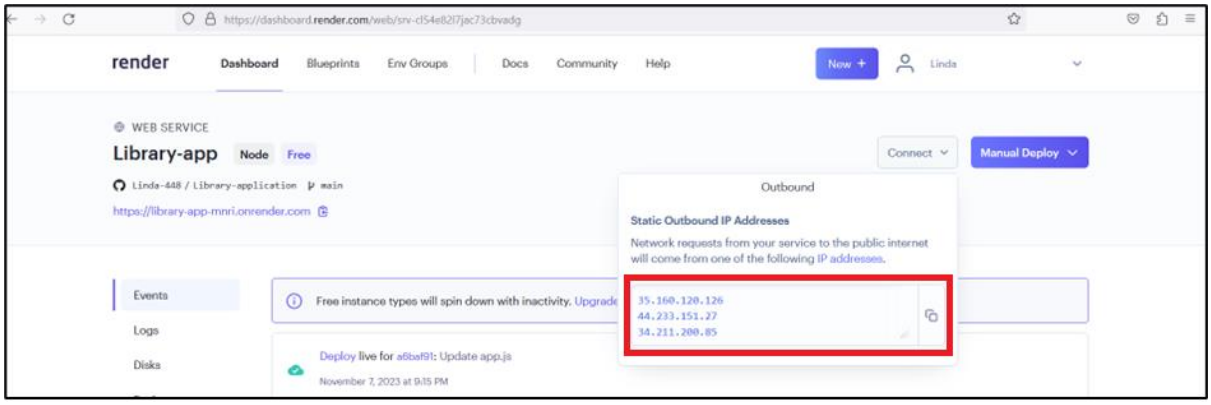


Figure 10.27: IP Addresses on Render Dashboard Page

17. To update the IP address, navigate to the MongoDB cloud account. The **Database Deployments** page is active.

    On the left, click **Network Access**.
    On the **Network Access** page, click **Edit** as shown in Figure 10.28.

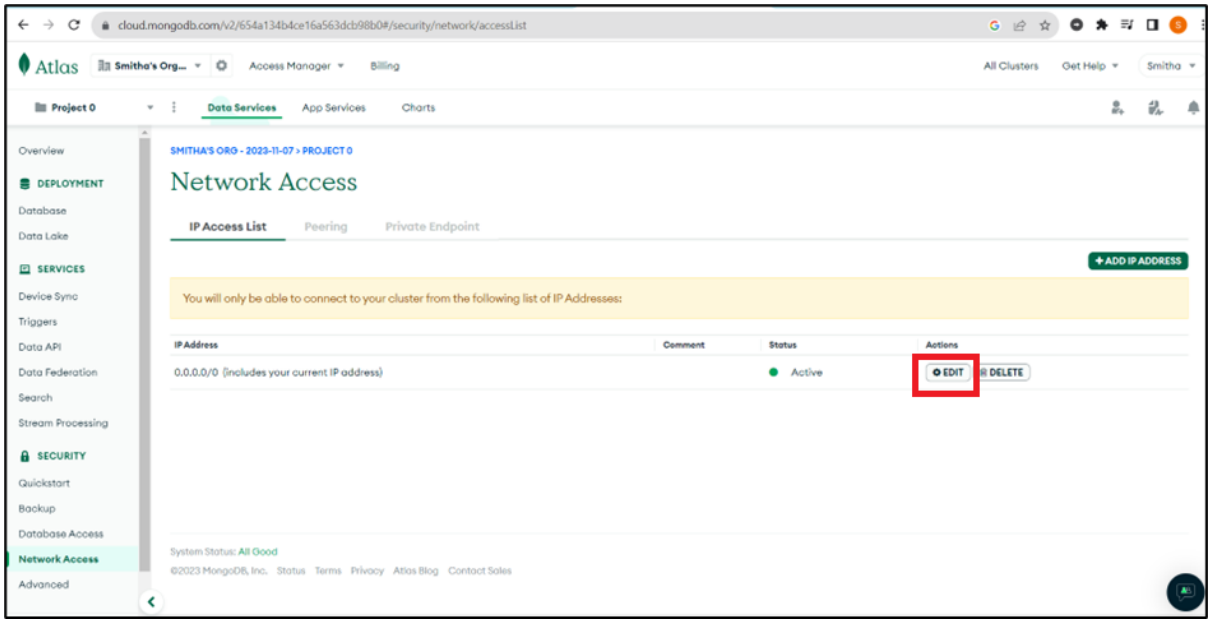


Figure 10.28: Network Access Page

18. To edit the IP address from 0.0.0.0/0 to 35.160.120.126, on the **Edit IP Access List Entry** page, in the **Access List Entry** box, type **35.160.120.126**.

    To specify a comment, in the **Comment** box, type **IP of Render**, and click **Confirm** as shown in Figure 10.29.

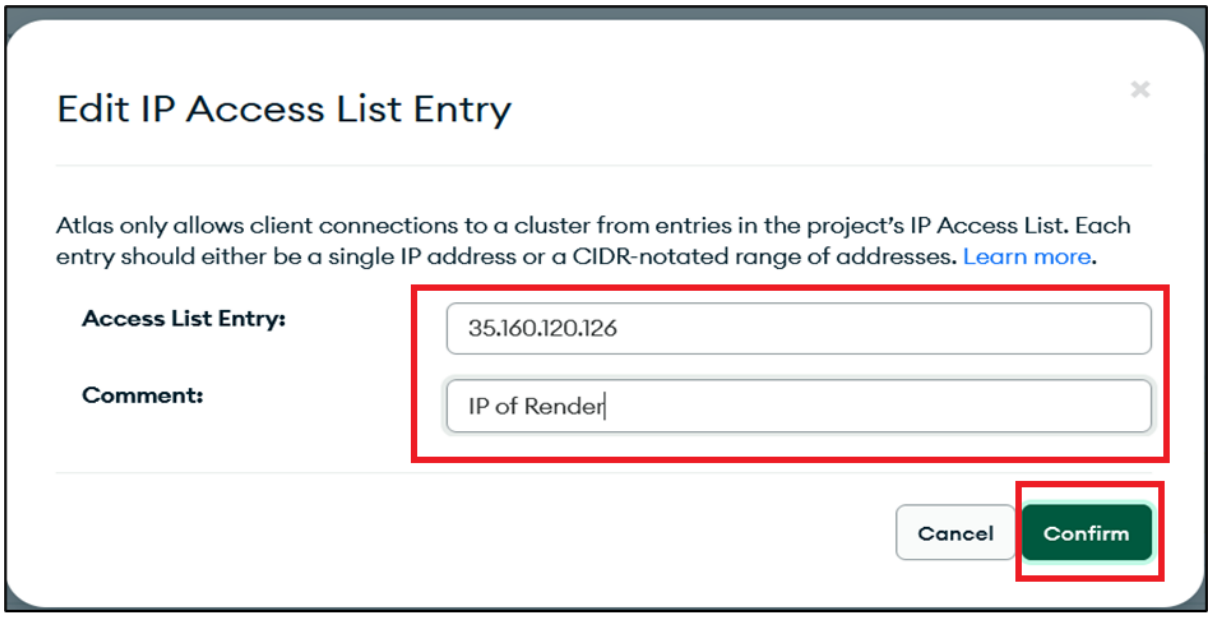


Figure 10.29: Edit IP Access List Entry Page

19. The IP address is updated as shown in Figure 10.30.

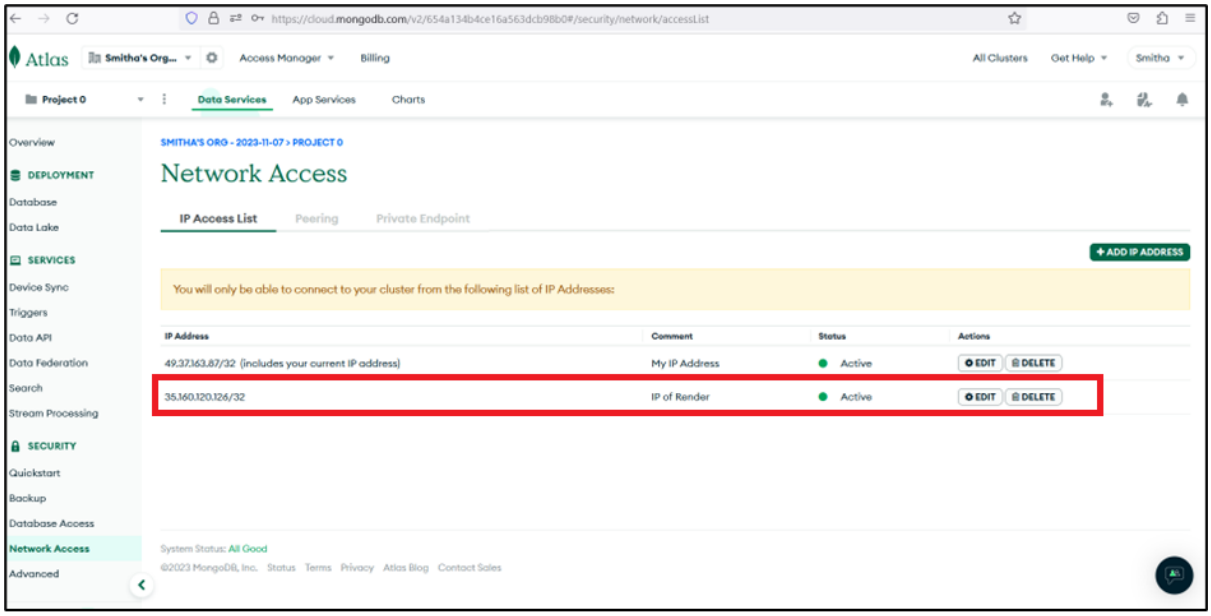


Figure 10.30: IP Address Updated on Network Access Page

20. To view the status of the **Library-app** project, navigate to **Render**, and click **Dashboard** as shown in Figure 10.31.

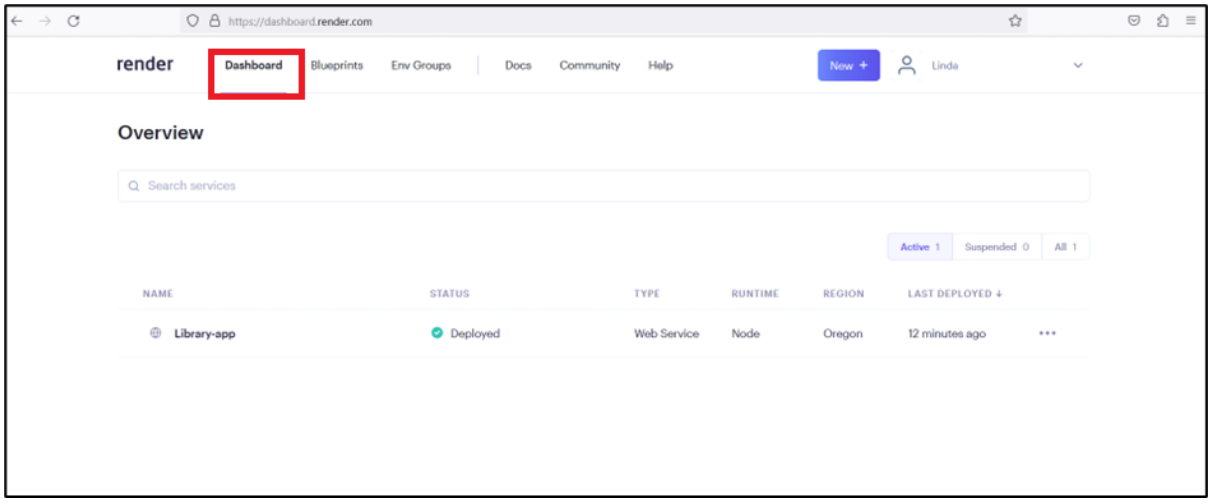


Figure 10.31: Project Status on Render Dashboard Page

21. To copy the URL, on the **Render Dashboard** page, click **Library-app**, select the URL, and press **Ctrl + C** as shown in Figure 10.32.

    The URL, https://library-app-mnri.onrender.com, is copied to the clipboard.

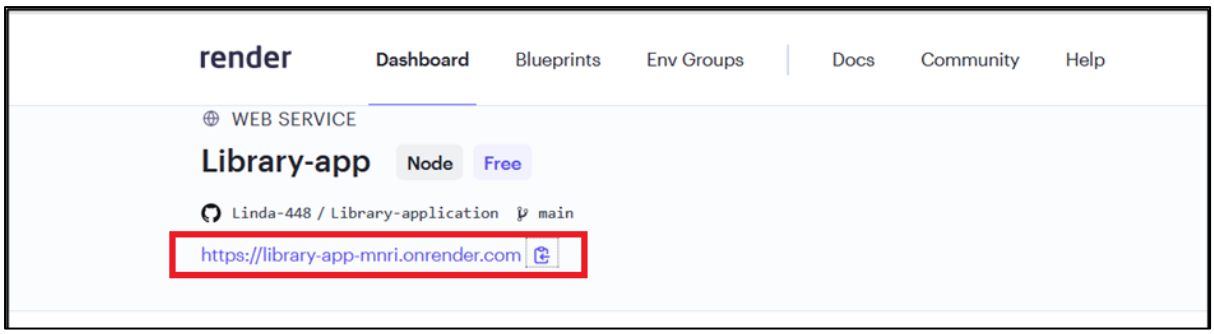


Figure 10.32: URL on Render Dashboard Page

10.6 Accessing the Library Application in Web Browser

The library application is now deployed and is ready to be accessed in the Web browser using the URL.

Perform the given steps:

1. Open the Web browser and paste the copied URL. The home page of the library application is displayed.

   To create a new member, click **Sign_up for Member User** as shown in Figure 10.33.

Figure 10.33: Home Page of Library Application

2. The sign up form for a new library member is displayed.

   To provide the username, in the **Username** box, type **Richard**.

To provide the password, in the **Password** box, type **1234**, and click **Sign-UP** as shown in Figure 10.34.

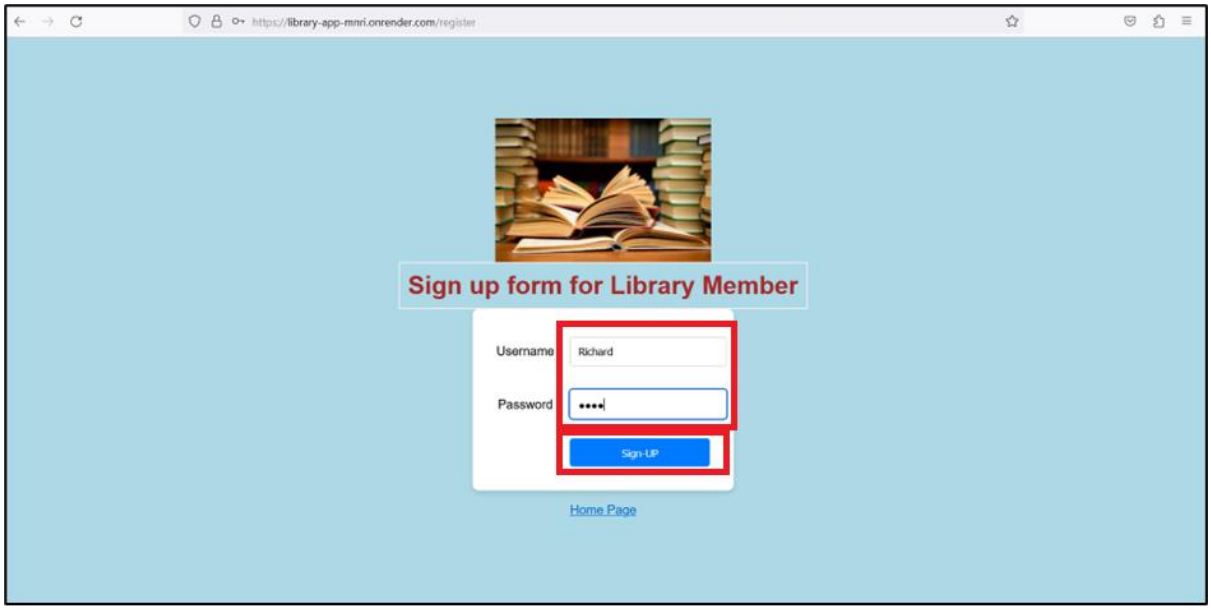


Figure 10.34: Sign Up Form for Library Member

3. To login as an administrator, on the home page, click **Administrator Login** as shown in Figure 10.35.

Figure 10.35: Home Page of Library Application

4. To provide the administrator username, on the **Admin Login** page, in the **Username** box, type **Admin**.
   To provide the password, in the **Password** box, type **12345**, and click **Login** as shown in Figure 10.36.

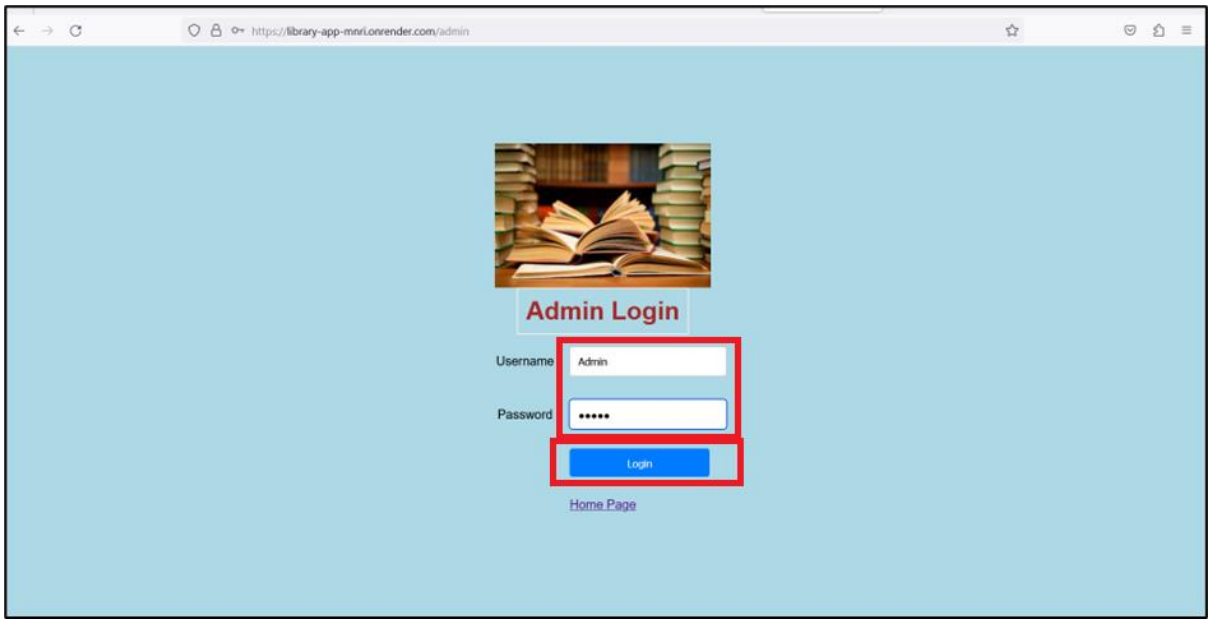


Figure 10.36: Admin Login Page of Library Application

5. To add the new book details, on the **Create Book Detail** page, type the given details, and click **Add Book** as shown in Figure 10.37.
   In the **BookID** box, type **1211**.
   In the **Book Name** box, type **The Secret of The House On The Hill**.
   In the **Author Name** box, type **Julia Harris**.
   In the **Price** box, type **$10**.
   In the **Age Group** box, type **15+**.
   In the **Book Type** box, type **Thriller**.
   The book details are added in the database.

   Add more book details, if required or click **Logout** to return to the home page.

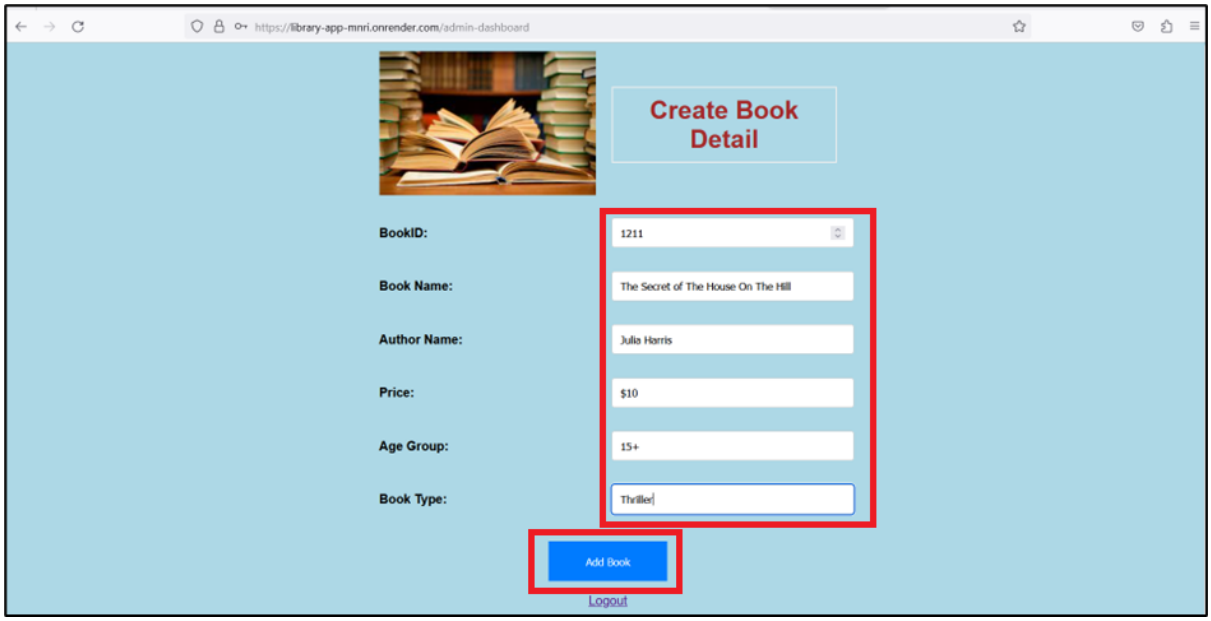


Figure 10.37: Create Book Detail Page of Library Application

6. To login as an existing member, on the **Library Home** page, click **Member Login** as shown in Figure 10.38.

Figure 10.38: Library Home Page

7. To provide the existing username, in the **Username** box, type **Richard**. To provide the password, in the **Password** box, type **1234**, and click **Login** as shown in Figure 10.39.

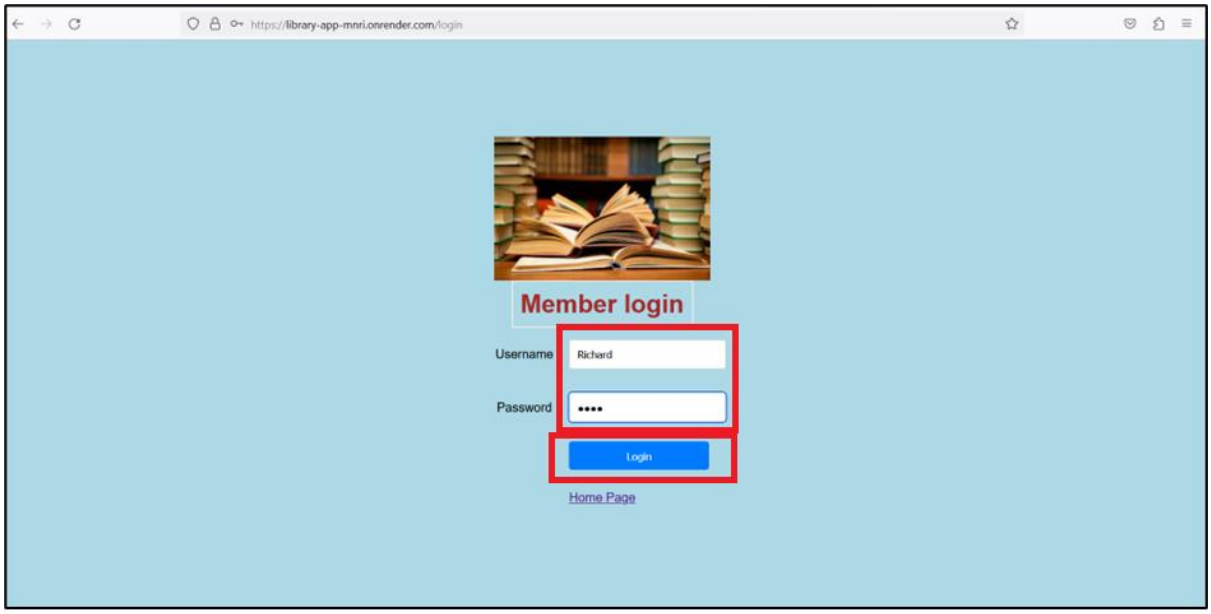


Figure 10.39: Member Login Page

8. On the **List of Books** page, the list of available books are displayed for the member as shown in Figure 10.40.

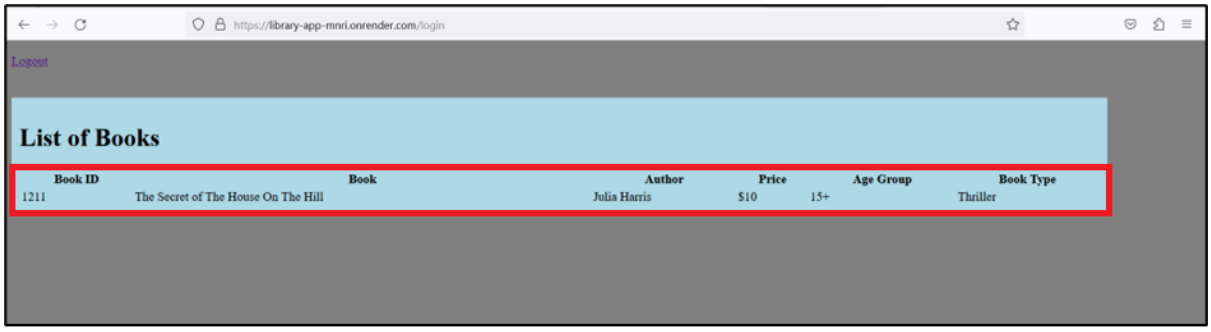


Figure 10.40: Book List on List of Books Page

9. To return to the home page of the library application, on the **List of Books** page, click **Logout** as shown in Figure 10.41.

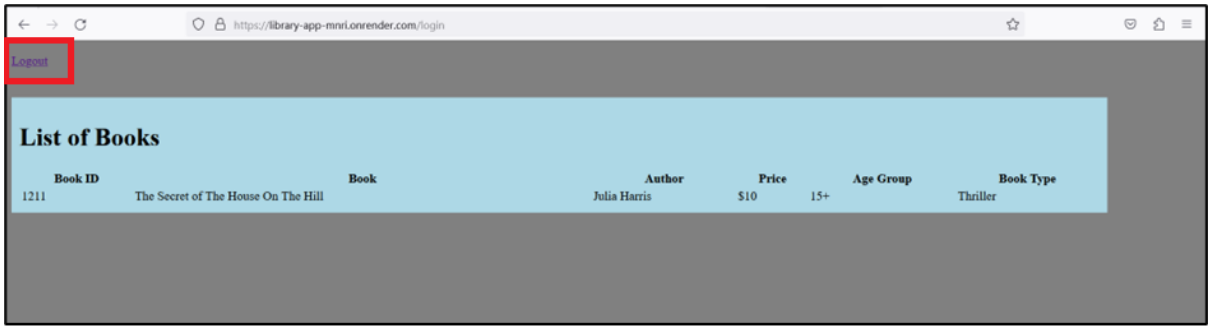


Figure 10.41: Library Home Page

10.7 Summary

- The connection string created in the MongoDB cloud database connects the Web application to MongoDB cloud.
- The Node.js packages are installed in the Web application using the `npm` commands.
- Some of the important tasks in Web application development are:
  - Setting up the view engine and middleware
  - Defining the local strategy and routes
  - Serializing and deserializing user functions
  - Listening for requests
- Schemas and stylesheets should be created in the respective Web application folders.
- The Web application can be uploaded to the GitHub repository for better collaboration and version control.
- Render can be used to deploy the Web applications on cloud.

Test Your Knowledge

1. Arrange the steps to create a simple Web application in sequence.

    i. Create a Web application in Node.js
    ii. Connect Web application to database
    iii. Upload the application to the GitHub Repository
    iv. Deploy the application on Render
    v. Access the application on the Web browser

    a) ii, i, iii, iv, and v
    b) i, ii, iii, iv, and v
    c) v, iv, iii, ii, and i
    d) i, ii, iii, v, and iv

2. Which command will you use to create a new project in the application folder?
    a) `npm init`
    b) `npm init -y`
    c) `npm init -x`
    d) `npm`

3. Which package in Node.js is used to parse the body of HTTP POST requests?
    a) `encryption`
    b) `body`
    c) `parse`
    d) `body-parser`

4. Which package is used to perform authentication using username and password in Node.js applications?
    a) `passport-local`
    b) `local`
    c) `authentication`
    d) `ejs`

5. What is the purpose of the `npm install ejs` command?
    a) To install `EJS` run time environment
    b) To install JavaScript package
    c) To install the server
    d) To install the `EJS` library

Answers to Test Your Knowledge

1	a
2	b
3	d
4	a
5	d

Try It Yourself

1. Create a simple Web application for the **Passport Portal**. Perform the given operations and deploy the application on **Render**.
   - Add a new profile and refer to the parameters such as `Name`, `Age`, `Bank Name`, and `ID Proof`.
   - Delete a profile.
   - Update a profile.

   a) Import the necessary packages.
   b) Create a database named, `Passport`.
   c) Add the `Passport_Collection` collection to the `Passport` database and insert 3 documents as given in Table 10.2.

Name	Age	Bank Name	ID Proof
Alex	30	Bank of America	Green Card
John	21	Citigroup	Voter's Registration
Harry	26	U.S. Bancorp	Green Card

Table 10.2: Passport Profiles

   a) View the inserted documents.
   b) Update the `Name Harry` to `Charlie`.
   c) Delete a document with the `Name` as `Alex`.
   d) Display the results on the browser.
   e) Deploy the application on **Render**.